SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · SYSTEM DESIGN · ELITE + BATCH 1

Lesson 5 · Scaling, Load Balancers, DNS and Proxies

Everything that sits between a user's tap and your server. Told through one story: a single click on our short-link service sho.rt, followed from the DNS lookup to the server that answers, and what happens when each layer breaks.

MODULE
System Design · Part A
Foundations

TIME
About 85 minutes
plus 50 minutes of practice problems

YOU NEED
Lessons 2 and 4
the URL journey and QPS arithmetic

YOUR ROUTE TODAY

01 A DNS record goes empty
02 Scale up or scale out
03 Load-balancing algorithms
04 L4, L7 and health checks
05 **DNS, proxies and CDNs**

LESSON 5 AT A GLANCE

What you will be able to do

Scale up vs scale out
When each is right; why horizontal needs stateless servers.

Pick an algorithm
Round robin, weighted, least connections, IP hash, consistent hashing.

L4 vs L7
What each kind of balancer can and cannot see.

Health checks
How long a dead server gets traffic; the "checks took the site down" trap.

DNS and TTL
Stub to authoritative; use TTL to plan an IP change.

Proxies and CDNs
Forward vs reverse, the API gateway, and when a CDN is worth it.

Subject / module	System Design / Part A: Foundations
Difficulty	Beginner. You need Lesson 2's request journey (DNS, TCP, TLS, HTTP) and Lesson 4's QPS arithmetic. Nothing else.
Prerequisites	Lessons 1 to 4. Lesson 2 walked through "what happens when you type a URL"; this lesson zooms into the DNS step and everything between the user and your server. Lesson 4 sized sho.rt at 7,716 redirects/s at peak and about 9 app servers.
Next lesson	Lesson 6: Caching
Companies covered	Amazon, Microsoft, Google, Uber, Flipkart, Atlassian, Adobe, Swiggy, Zomato, PhonePe, Razorpay
Commands and configs	nginx, dig, curl, AWS target-group settings. No programming language is required.
Checklist	☐ vertical vs horizontal ☐ stateless servers ☐ round robin, least connections, consistent hashing ☐ L4 vs L7 ☐ health checks ☐ DNS resolution ☐ forward vs reverse proxy ☐ CDN basics
Citations	[S#] = a page opened in full; [X#] = text seen only in search results (lists on the Sources pages). Practice problems and quiz questions are original and unlabeled.

SOURCE-CONFIRMED
A dated first-person report naming the company and the question, Oct 2024 to Oct 2026, opened in full.

CANDIDATE-REPORTED
Compilations, aggregators, forum threads, older than 24 months, undated, or seen only in search extracts.

PATTERN-ONLY
Commonly asked, but no company source, so no company name is attached.

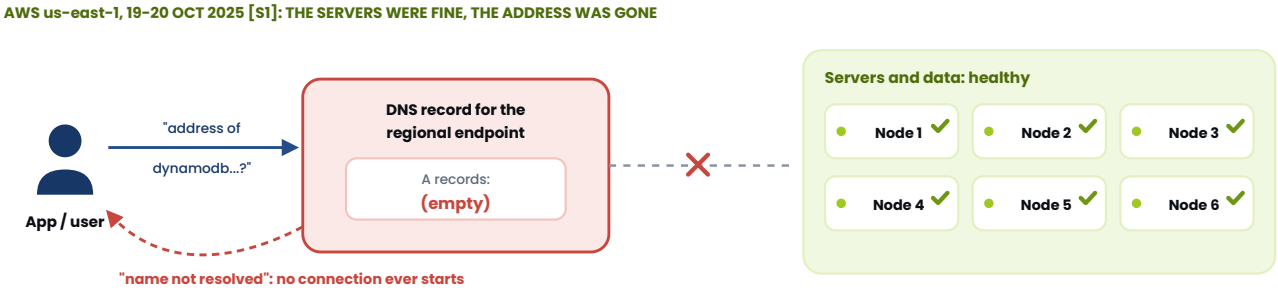
AN HONEST NOTE ON THE EVIDENCE

These topics are asked often, but mostly as one-liners inside bigger rounds, so few reports write them down. Only **two** dated first-person reports fall inside the window: Amazon (Chennai, April 2025) asked "what is a load balancer" [S20], and Google (Customer Solutions Engineer, Singapore, January 2026) asked to explain DNS [S21]. Adobe, Razorpay, Atlassian, Microsoft and Zomato items are Candidate-reported. No report on these topics was found for Uber, Swiggy or PhonePe software-engineer rounds inside the window, and we say so rather than invent one.

19 OCTOBER 2025, NORTHERN VIRGINIA

The servers were fine. The address was gone.

Amazon DynamoDB in AWS's us-east-1 region started failing. The servers and the data were healthy. What broke was a **DNS record**.



Cause: two copies of an automation tool raced; a delayed one applied an old plan; the other's clean-up deleted that plan, leaving the record empty.
Errors 11:48 PM to 2:40 AM (PT). Later on 20 Oct, flapping NLB health checks removed working capacity until failover was switched off at 9:36 AM.

The building is open, the staff are at their desks, and someone has quietly removed the street from Google Maps.

Two copies of an automation tool raced. A delayed one applied an old plan, the other's clean-up step deleted that plan, and the regional endpoint's record was left empty. In AWS's words, **"all IP addresses for the regional endpoint were immediately removed"** [S1]. Error rates stayed high from 11:48 PM to 2:40 AM (Pacific time). Later on 20 October, Network Load Balancer health checks that kept flipping between failing and healthy took working capacity out of service, until engineers switched automatic failover off at 9:36 AM [S1].

PAUSE AND PREDICT

If every server is healthy but DNS returns no address, what does a user's app see?

Answer: A **name-resolution error**. With no address, no connection ever starts.

INTERVIEW MODE: SAY IT LIKE THIS

- Users reach servers through DNS, load balancers and proxies. Each layer adds scale and safety, and each layer can fail on its own.
- A good design explains how traffic is spread, how failures are detected, and what happens when the routing layer itself breaks.

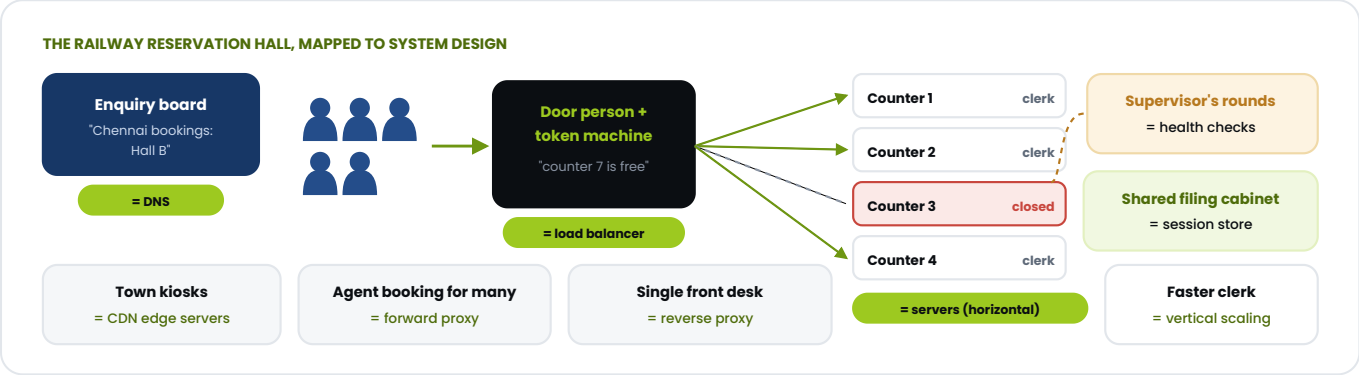
MEMORY HOOK: THE ONE SENTENCE

"Every layer in front of the server is also a thing that can break." This lesson is about adding layers on purpose, and knowing their failure modes.

BEFORE THE JARGON

The railway reservation hall

Picture a big railway reservation hall before online booking. Hundreds of people arrive. Here is how the hall copes.



One faster clerk (better computer, faster printer) is **vertical scaling**. It helps, up to a point. No clerk can serve 10,000 people an hour, and if that one clerk falls ill, the hall closes.

More counters is **horizontal scaling**. Ten ordinary clerks beat one superhuman clerk, and one clerk on chai break does not close the hall.

The **token machine and door person** saying "counter 7 is free" is the **load balancer**. "Next counter in order" is round robin; "shortest queue" is least connections. The **supervisor** checking every few minutes that each clerk is working is the **health check**: a clerk asleep at the counter gets their window closed.

The **enquiry board** telling you which hall handles your train is **DNS**: you know the name ("Chennai Central bookings"); the board tells you where to physically go. The **agent** who books for many passengers is a **forward proxy** (acts for passengers); the **single front desk** that receives every passenger and passes forms to clerks in the back office is a **reverse proxy** (acts for clerks). **Town kiosks**, so you need not travel to the main station, are the **CDN**.

THE ONE CONDITION THAT MAKES THE HALL WORK

Any clerk must serve any passenger

THE KEY CONDITION: THE STATELESS RULE

Ten counters only work if **any clerk can serve any passenger**. If clerk 3 keeps your half-filled form in her drawer, you must return to counter 3, and if she goes home, your form is gone. Keep the paperwork in a **shared cabinet** (a database or cache), not in one clerk's drawer.

MEMORY HOOK: THE HALL IN ONE LINE

Board tells you where, **door** picks a counter, **supervisor** closes broken counters, **cabinet** keeps the paperwork.

WHAT GOES WRONG

Where the analogy breaks. A hall serves people one at a time; a server handles hundreds at once, so "queue length" really means connections in flight. The enquiry board changes instantly when repainted, but DNS answers are cached across the internet for a time you choose (the TTL), so changes spread slowly. And a kiosk cannot sell you a fresh seat by itself: a CDN serves copies of unchanging content; anything personal or live still goes to the main hall (the origin).

TINY CHALLENGE

In the hall, what is the single point of failure in the vertical version, and in the horizontal version?
Answer: Vertical: the one clerk. Horizontal: the door person, unless there are two of them. Hold that thought for Concept 7.

HALL	SYSTEM DESIGN
Faster clerk	Vertical scaling (bigger machine)
More counters	Horizontal scaling (more machines)
Door person, token machine	Load balancer and its algorithm
Supervisor's rounds	Health checks
Enquiry board	DNS
Booking agent for passengers	Forward proxy
Single front desk for clerks	Reverse proxy
Town kiosks	CDN edge servers
Shared filing cabinet	External session store (database, cache)

INTERVIEW MODE: SAY IT LIKE THIS

- Vertical scaling buys a bigger machine; horizontal scaling adds machines behind a load balancer, and only works when servers are stateless.
- DNS turns a name into an address, proxies sit in the path for clients or for servers, and a CDN serves copies of content close to users.

EVERY WORD, ONE LINE (1 OF 2)

Vocabulary primer

Every term is defined before it is used. Terms from Lessons 2 and 4 (RTT, QPS, peak, latency, p99, headroom, TLS) are reused without re-definition.

Scaling
Changing a system so it can handle more load.

Horizontal scaling (scale out)
Adding more machines and spreading the load across them.

SPOF
Single point of failure: one component whose failure stops the whole system.

Stateful server
Keeps per-user data (such as a login session) in its own memory or disk.

Sticky session / session affinity
Sending the same user to the same server every time.

Load-balancing algorithm
The rule the LB uses to pick a server for each request or connection.

Weighted round robin
Round robin where a stronger server gets more turns, in proportion to its weight.

Static vs dynamic algorithm
Static ignores the servers' current state (round robin); dynamic looks at it (least connections) [S10].

Mod N hashing
Pick server number hash(key) mod N, where N is the number of servers.

Hash ring
The picture for consistent hashing: hash values arranged in a circle.

OSI model
A seven-layer reference model of networking; L3 network (IP), L4 transport (TCP, UDP), L7 application (HTTP).

L4 load balancer
Balances using IP addresses and ports only; does not read HTTP [S8, S12].

TLS termination
Decrypting HTTPS at the LB or proxy so the backend receives plain HTTP (or re-encrypted traffic).

Vertical scaling (scale up)
Making one machine bigger: more CPU, memory or faster disk.

Node / instance / backend / target
One server in a group. AWS calls servers behind a balancer targets [S12]; nginx calls the group an upstream [S6].

Stateless server
Keeps no per-user data between requests; anything that must persist lives in a shared store [S15].

Session
Data that ties a user's requests together, such as "logged in as Priya".

Load balancer (LB)
Spreads incoming requests across several servers and avoids unhealthy ones [S12].

Round robin
Pick servers in turn: 1, 2, 3, 1, 2, 3...

Least connections
Pick the server with the fewest active connections right now [S6].

Hash function
Turns any input (an IP, a URL) into a number that looks random but is always the same for the same input.

Consistent hashing
Adding or removing one server moves only a small share of keys [S14].

Virtual node
One of many points on the ring that belong to the same real server, to even out load [S14].

TCP / UDP
TCP gives a reliable ordered connection; UDP sends independent packets with no connection.

L7 load balancer
Reads the HTTP request (path, headers, cookies) and routes on it; a proxy with two TCP connections [S8].

Health check
A periodic test the LB runs to decide if a server should receive traffic [S12, S13].

EVERY WORD, ONE LINE (2 OF 2)

Health checks, DNS, proxies

Active vs passive health check
Active: the LB sends probe requests. Passive: the LB watches real traffic for failures [S6].

Threshold, interval, timeout
How many checks in a row decide a state change, how often checks run, how long to wait for an answer [S13].

Connection draining
Letting in-flight requests finish before removing a server; AWS calls it deregistration delay (default 300 s) [S33].

Autoscaling
Adding or removing servers automatically based on load.

DNS
Domain Name System: turns names such as sho.rt into IP addresses.

Stub resolver
The small DNS client in your OS that hands lookups to a recursive resolver [S3].

Root, TLD, authoritative nameservers
Root points to the TLD (such as .com), the TLD points to the domain's authoritative server, which holds the records [S2].

TTL
Time to live: how many seconds a DNS answer may be cached before it should be discarded [S3].

Proxy
A server between a client and a server that passes requests along.

Reverse proxy
Sits in front of servers and receives clients' requests for them [S4].

Origin
The server that holds the original content [S5].

Edge server / PoP
A CDN server near users; a PoP (point of presence) is a location that holds edge servers.

X-Forwarded-For
An HTTP header in which proxies record the original client's IP address.

Liveness vs readiness
Liveness: "is the process alive?" Readiness: "can it serve traffic right now?" (common usage, made popular by Kubernetes).

Failover
Moving traffic from a failed component to a working one.

Flapping
A health state that keeps switching between healthy and unhealthy [S1].

Availability zone (AZ)
A separate data-centre location inside one cloud region, with independent power and networking.

Domain name
A human-readable name such as api.sho.rt.

Recursive resolver
The DNS server (often your ISP's, or a public one such as 1.1.1.1) that finds the answer for you and caches it [S2].

Record types
A (IPv4 address), AAAA (IPv6), CNAME (alias to another name) [S3], NS (which nameservers are authoritative).

Anycast
Many servers in different places share one IP address; the network delivers you to a nearby one [S2].

Forward proxy
Sits in front of clients and talks to the internet for them [S4].

API gateway
A reverse proxy specialised for APIs: routing by path, authentication, rate limiting.

CDN
Content delivery network: many edge servers that cache content close to users [S5].

Cache hit / miss
Hit: the answer was already stored. Miss: it had to be fetched from the origin [S5].

INTERVIEW MODE: SAY IT LIKE THIS

- I use "load balancer" for the component, "algorithm" for its rule, "health check" for how it detects bad servers, and "target" or "backend" for the servers behind it.
- I say "L4" when the balancer sees only IPs and ports, and "L7" when it reads HTTP.

CONCEPT 1 · VERTICAL VS HORIZONTAL SCALING

One big box, or many small ones?

Ten steps, bottom-up. The running example is sho.rt from Lessons 1 to 4: **7,716 redirects per second at peak**, an assumed **2,000 simple requests per second per app server**, and about **9 app servers** with headroom.

The problem. One server handles about 2,000 redirects per second. Peak is 7,716. Something has to give. You have two levers.

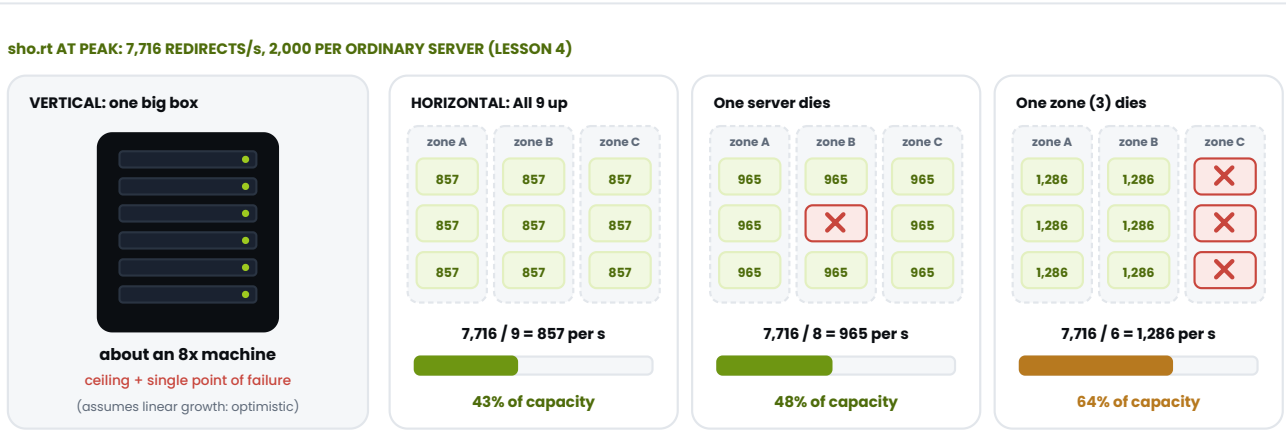
Vertical (scale up): a bigger machine

No code changes, no new components. But there is always a largest machine you can buy, the price per unit of capacity tends to rise at the top end, resizing often needs a restart, and the one big machine is still a single point of failure.

Horizontal (scale out): more ordinary machines

Put a load balancer in front. Capacity grows by adding machines, and one machine dying removes only a slice. The cost: a load balancer, health checks, and the rule that servers must be stateless (Concept 2).

AWS's reliability guidance says it directly: **"Replace one large resource with multiple small resources to reduce the impact of a single failure on the overall workload"** [S16].

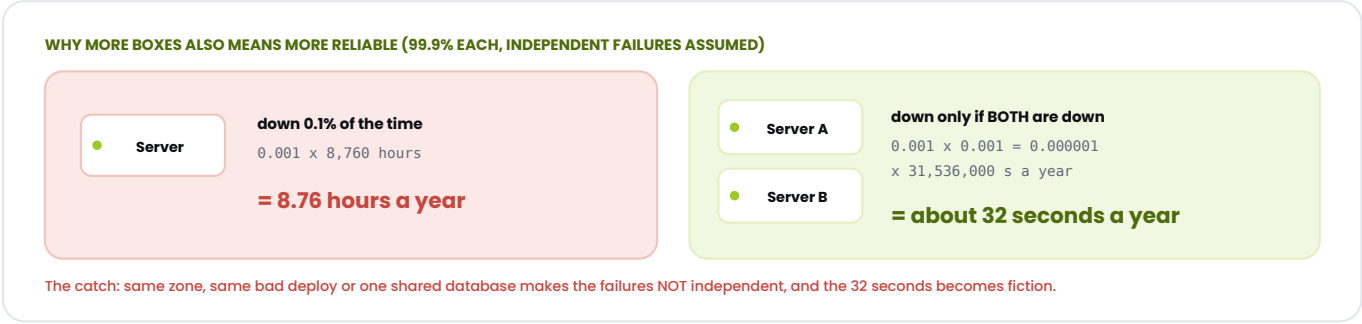


Spread across 3 zones, even losing a whole zone leaves each surviving server at 64%, still under its 2,000/s limit.

OPTION	CALCULATION	RESULT
Vertical: one big box	7,716 / 2,000 = about 3.9x an ordinary server, and 2x that for 50% headroom	About an 8x machine, assuming capacity grows linearly with size (optimistic)
Horizontal: 9 ordinary boxes	7,716 / 9	857 per second each, 43% of capacity
Horizontal, one server dies	7,716 / 8	965 per second each, 48%
Horizontal, one zone of 3 servers dies	7,716 / 6	1,286 per second each, 64%

CONCEPT 1 · THE AVAILABILITY SUM

More boxes also means more reliable



Assume each server is up 99.9% of the time and failures are independent (an assumption; see the trap below). One server is down 0.1% of a year: 0.001 x 8,760 hours = **8.76 hours a year**. With two servers where either one is enough, both are down only 0.001 x 0.001 = 0.000001 of the time: 0.000001 x 31,536,000 s = **about 32 seconds a year**. That is the maths behind "no single point of failure".

When vertical is the right answer. When the load fits comfortably on one machine with headroom, when the component is hard to split (a single relational primary, Lesson 3), or early in a product's life when simplicity matters more than redundancy. Many real systems scale the **database vertically** for a long time and scale the **stateless app tier horizontally**.

WHAT GOES WRONG

(1) The independence assumption. If all 9 servers sit in one zone, share one bad deploy, or depend on one database, they fail together and the 32-second figure is fiction. **(2)** Scaling out the app tier while the database stays one box simply moves the bottleneck (Lesson 7).

Just for fun. Adding ten more billing counters does not help if they all share one printer that jams.

INTERVIEW MODE: SAY IT LIKE THIS

- "Vertical scaling makes one machine bigger; it is simple but has a ceiling and stays a single point of failure."
- "Horizontal scaling adds machines behind a load balancer; it needs stateless servers but gives both capacity and redundancy."
- "At 7,716 requests per second and 2,000 per server, I would run about 9 servers across zones, so losing a whole zone still leaves us at about 64% load."

TINY CHALLENGE

12 servers spread evenly over 3 zones, same peak. Per-server load if one zone fails? Answer: 7,716 / 8 = about 965 per second, 48%.

MEMORY HOOK: UP VS OUT

Up = one taller building (there is a height limit, and one fire takes it all). **Out** = more buildings on the street.

CONCEPT 2 · STATELESS SERVERS

Any server must be able to serve any request

Horizontal scaling assumes any server can handle any request. That breaks the moment a server remembers something about a user that the others do not know.

A **stateless** server keeps nothing between requests that another server would need. The Twelve-Factor App guidance is blunt: processes are **"stateless and share-nothing"**, and **"any data that needs to persist must be stored in a stateful backing service, typically a database"** [S15]. Memory can still be used as "a brief, single-transaction cache" [S15], meaning inside one request.

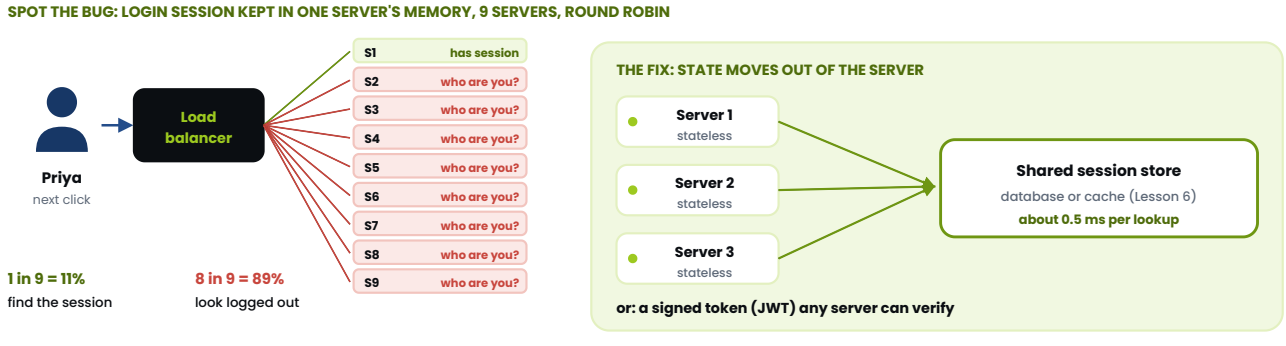
WHERE SESSION DATA GOES	HOW IT WORKS	TRADE-OFF
Shared session store	Sessions in a database or an in-memory store (Lesson 6) that every server reads	One more component to run; one network round trip per request (about 0.5 ms in a data centre, Lesson 4)
Signed token on the client	The client sends a signed token (such as a JWT, Lesson 2) that any server can verify	Hard to revoke before expiry; token size travels with every request
Sticky sessions	The LB keeps sending a user to the same server	Uneven load; sessions die with the server; Twelve-Factor calls them "a violation… should never be used or relied upon" [S15]

Worked example: spot the bug. sho.rt adds logged-in dashboards. Version 1 stores the login session in the app server's memory. With 9 servers and round robin, what fraction of a user's next requests land on the server that has their session?

PAUSE AND PREDICT

Pause and predict before reading the picture.

Answer: Only **1 in 9, about 11%**. The other 89% reach a server that has never heard of the user, so the user appears logged out at random.

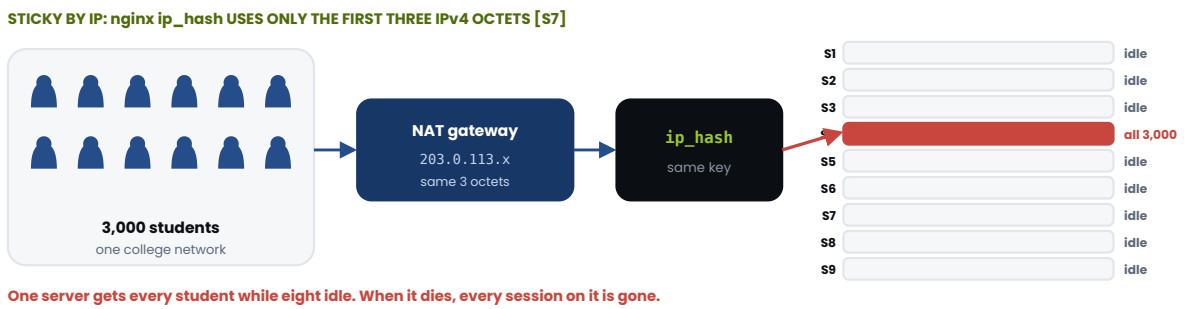


Just for fun. Users describe this bug as "the site keeps logging me out", and developers describe it as "works on my machine", because on a laptop there is only one server.

CONCEPT 2 · WHY STICKY SESSIONS ARE A CRUTCH

Stickiness hides the bug and creates two more

nginx's `ip_hash` makes clients sticky by hashing their IP address [S6]. Its documentation adds a detail most people miss: for IPv4, **"the first three octets of the client IPv4 address... are used as a hashing key"** [S7]. So every user whose address shares the first three numbers, for example everyone behind one college's network address translation (NAT) gateway, lands on the same server.



If 3,000 students sit behind one gateway, one server gets all 3,000 while eight others idle; when it dies, every session on it is gone.

WHAT GOES WRONG

Hidden state is the usual culprit: files uploaded to local disk, an in-memory rate-limit counter (Lesson 1's rate limiter), a scheduled job that runs on "server 1". Each breaks when there are two servers.

INTERVIEW MODE: SAY IT LIKE THIS

- "My app servers are stateless: sessions live in a shared store or a signed token, so any server can serve any request."
- "Sticky sessions work as a stopgap, but they unbalance load and lose sessions when a server dies."
- "Making servers stateless is what lets me add or remove them freely, including with autoscaling."

TINY CHALLENGE

4 servers and in-memory sessions: what percentage of follow-up requests look logged out?
Answer: 3 in 4, 75%.

MEMORY HOOK: DRAWER VS CABINET

Never keep the passenger's form in **one clerk's drawer**. Put it in the **shared cabinet**.

CONCEPT 3 · WHAT A LOAD BALANCER DOES

One address in front, many servers behind

Clients know one address. Behind it are 9 servers. Something must pick a server for every request (or connection) and skip broken ones.

AWS's description is a good definition: a load balancer **"distributes incoming traffic across targets"**, **"monitors the health of its registered targets and ensures that it routes traffic only to healthy targets"** [S12]. The choice of server is the **algorithm**. Algorithms are **static** (they ignore server state) or **dynamic** (they look at it) [S10].

Round robin hands out requests in turn. It is nginx's default when no method is configured [S6]. It is simple and fair if the servers are equal and the requests cost about the same.

Weighted round robin gives stronger servers more turns. nginx's documented example [S6]:

```
upstream myapp1 {
    server srv1.example.com weight=3;
    server srv2.example.com;
    server srv3.example.com;
}
```

The weight defaults to 1 [S7], so the weights are 3, 1 and 1, a total of 5. The documentation says every 5 new requests split as 3 to srv1, 1 to srv2 and 1 to srv3 [S6]: **60%, 20%, 20%.**

MEMORY HOOK: WEIGHTS ARE NOT PERCENTAGES

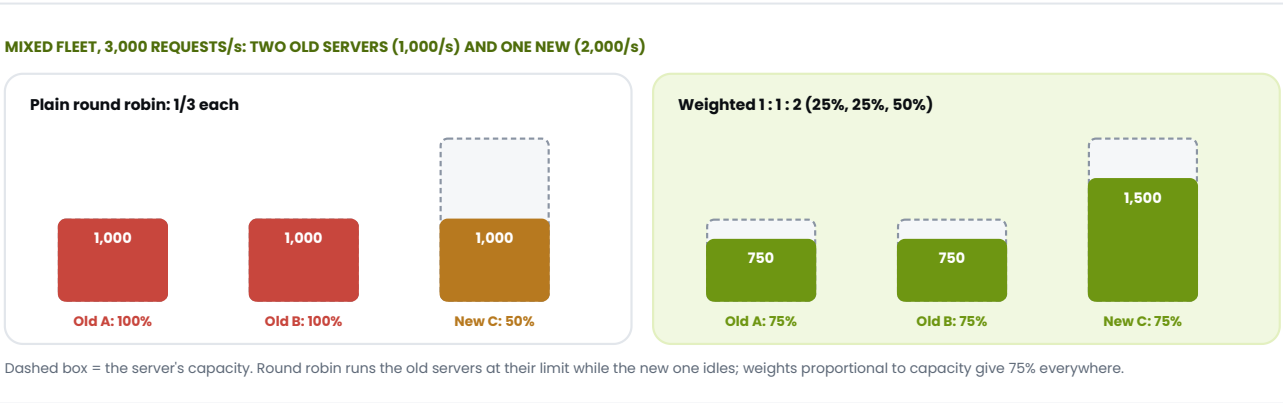
share = weight / sum of weights. 3 : 1 : 1 is 3/5, 1/5, 1/5.

PAUSE AND PREDICT

Weights 3, 1, 1 and 1,000 requests a second. How many go to srv1?

Answer: 600 (3/5 of 1,000).

Worked example: a mixed fleet. sho.rt has two old servers that handle 1,000 requests per second each and one new server that handles 2,000. Traffic is 3,000 requests per second.



Dashed box = the server's capacity. Round robin runs the old servers at their limit while the new one idles; weights proportional to capacity give 75% everywhere.

ALGORITHM	OLD SERVER A	OLD SERVER B	NEW SERVER C
Plain round robin (1/3 each)	1,000 rps = **100% load**	1,000 rps = **100%**	1,000 rps = 50%
Weighted 1 : 1 : 2 (shares 25%, 25%, 50%)	750 rps = **75%**	750 rps = **75%**	1,500 rps = **75%**

CONCEPT 3 · WHERE ROUND ROBIN FALLS SHORT

Round robin counts requests, not work

Plain round robin runs the old servers at their limit while the new one idles at half. Weights proportional to capacity even it out at 75% everywhere.

WHAT GOES WRONG

Round robin counts requests, not work. If one request costs 2 seconds and another costs 20 milliseconds, "equal turns" is not equal load. That is the next concept.

Just for fun. Round robin is the class monitor who gives every student exactly one question to answer, including the one who got the three-page derivation.

INTERVIEW MODE: SAY IT LIKE THIS

- "A load balancer gives clients one address, spreads requests over healthy servers, and removes failed ones."
- "Round robin is the default and works when servers and requests are similar; weighted round robin handles servers of different sizes, with share equal to weight over total weight."

TINY CHALLENGE

Weights 2, 2 and 4 with 8,000 requests per second: how many does each get? Answer: 2,000, 2,000 and 4,000.

STATIC VS DYNAMIC, IN ONE LINE

Static algorithms (round robin, weighted round robin, IP hash) decide without looking at the servers. **Dynamic** algorithms (least connections) look at each server's current state before choosing [S10]. Next: the most common dynamic one.

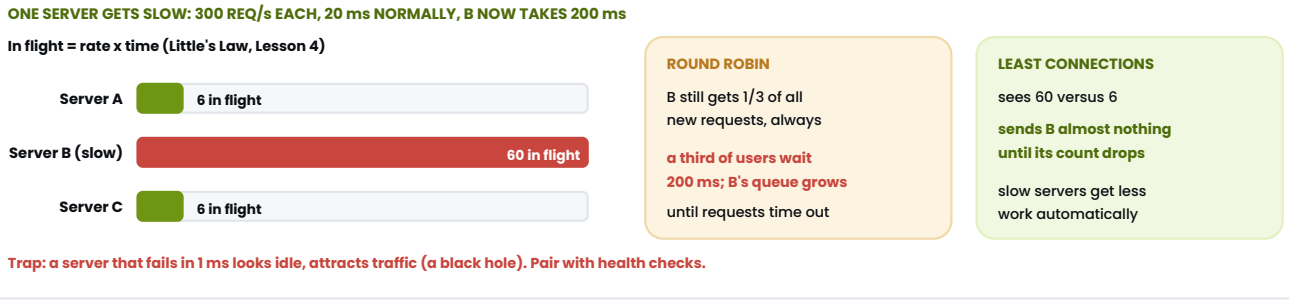
CONCEPT 4 · LEAST CONNECTIONS

Send work to the server with the shortest queue

Some requests are slow, and some servers get temporarily slow: a long garbage-collection pause (the runtime briefly stops to free memory), a noisy neighbour (another program on the same physical machine hogging resources), a cold cache. Round robin keeps feeding them anyway.

Least connections sends each new request to **"the server with the least number of active connections"** [S7]. nginx notes it helps when some requests take longer to complete [S6]. Why does "connections in flight" measure load? **Lesson 4's Little's Law:** requests in flight = arrival rate x time each request takes. A server that slows down holds each request longer, so its in-flight count rises, and least connections automatically sends it less new work.

HAProxy's documentation adds a nuance: it recommends `leastconn` for long-lived connections, such as **"SQL databases, gRPC streams, LDAP"**, and calls it poorly suited to short HTTP sessions [S9]. With very short requests, connection counts are small and noisy, so the benefit shrinks. Uber's on-host proxies used a close cousin, **least-pending requests**, for service-to-service traffic [S18].



WORKED EXAMPLE	SERVER A	SERVER B (SLOW)	SERVER C
In flight, normal (300 x 0.02 s)	6	6	6
In flight with round robin after B slows (300 x 0.2 s for B)	6	60	6
Share of new requests B gets with round robin		1/3 always	
What least connections does		sees 60 versus 6, sends B almost nothing until its count drops	

CONCEPT 4 · TWO TRAPS

The black hole and the new server flood

With round robin, a third of all users now wait 200 ms, and B's queue may keep growing until it times out. With least connections, B is starved of new work until it recovers.

WHAT GOES WRONG

(1) The black hole. A server that fails fast (returns errors in 1 ms) looks wonderfully idle to least connections and attracts more traffic. Health checks (Concept 7) must catch it. **(2) The flood.** A freshly added server has zero connections and can get flooded at once; some balancers ramp new servers up slowly (this is called **slow start**; check your balancer's documentation).

INTERVIEW MODE: SAY IT LIKE THIS

- "Least connections sends each request to the server with the fewest in-flight requests, so slow servers naturally get less work."
- "It shines for long-lived or uneven requests; for short uniform requests round robin is about as good."
- "A server that fails fast can look idle and attract traffic, so I pair it with health checks."

TINY CHALLENGE

A server receives 400 requests per second at 50 ms each. How many are in flight? Answer: 400 x 0.05 = 20.

MEMORY HOOK: SHORTEST QUEUE

Round robin is **"next counter"**. Least connections is **"shortest queue"**. A counter whose clerk says "no" instantly always has the shortest queue: that is the black hole.

SITUATION	GOOD DEFAULT
Equal servers, short uniform requests	Round robin
Servers of different sizes	Weighted round robin (weight proportional to capacity)
Uneven request times, long-lived connections (DB, gRPC streams, long polling)	Least connections (or least-pending requests)
Same key must reach the same server (cache affinity)	Hashing: consistent hashing (next concept)

CONCEPT 5 · HASHING: IP HASH AND MOD-N

Same key, same server

Sometimes you want the same key to reach the same server every time: the same user (stickiness) or, more usefully, the same cache key.

If every server in sho.rt keeps a local in-memory cache of hot links, round robin means each server must cache everything hot. Routing by short code means each server caches only its slice.

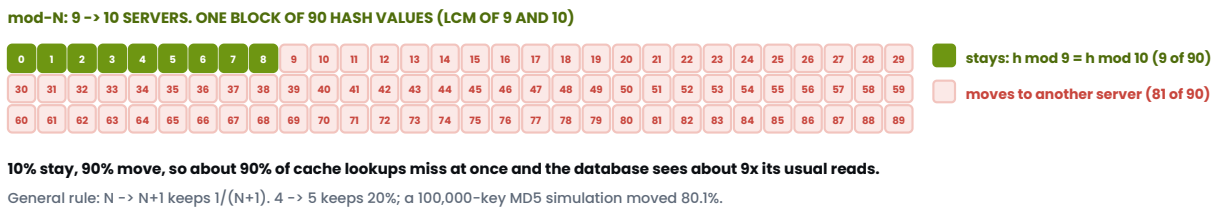
Step 1: mod-N hashing. Compute `server = hash(key) mod N`. Same key, same server. HAProxy suggests URI hashing (`balance uri`) for cache servers [S9].

Worked example: the mod-N trap. Lesson 4 sized the hot set at about 33 GB. With 9 servers hashed by short code, each caches about 33 / 9 = **3.7 GB**. With round robin, each would need all 33 GB for the same hit rate. Hashing wins.

PAUSE AND PREDICT

Now add a 10th server. What fraction of keys change server under mod-N?

Answer: About **90%**. See the grid.



10% stay, 90% move, so about 90% of cache lookups miss at once and the database sees about 9x its usual reads.

General rule: N -> N+1 keeps 1/(N+1). 4 -> 5 keeps 20%; a 100,000-key MD5 simulation moved 80.1%.

Derivation. A key stays put only if h mod 9 = h mod 10. In every block of 90 consecutive hash values (90 is the least common multiple of 9 and 10), exactly 9 values satisfy this (h = 0 to 8 in the first block). So 9 / 90 = 10% stay and 90% move. In general, going from N to N + 1 servers keeps **1 / (N + 1)** of keys and moves the rest. From 4 to 5 servers: 1 / 5 = 20% stay, 80% move. The source checked this with a simulation of 100,000 keys: **80.1% moved** (original simulation, MD5 hash).

WHAT GOES WRONG

Moving 90% of keys means 90% of cache lookups miss at once, and the database behind the cache suddenly receives **about 9 times** its usual read load.

Just for fun. Adding one server to improve things and instantly flattening your database is the system-design version of fixing a dripping tap with a hammer.

CONCEPT 5 · CONSISTENT HASHING

Put servers and keys on one ring

Karger and colleagues described it in a 1997 paper on web caching [S14, X1]. Their motivation was exactly this problem: with a classical hash, when the range changes, **"almost every item would be hashed to a new location"** [S14].

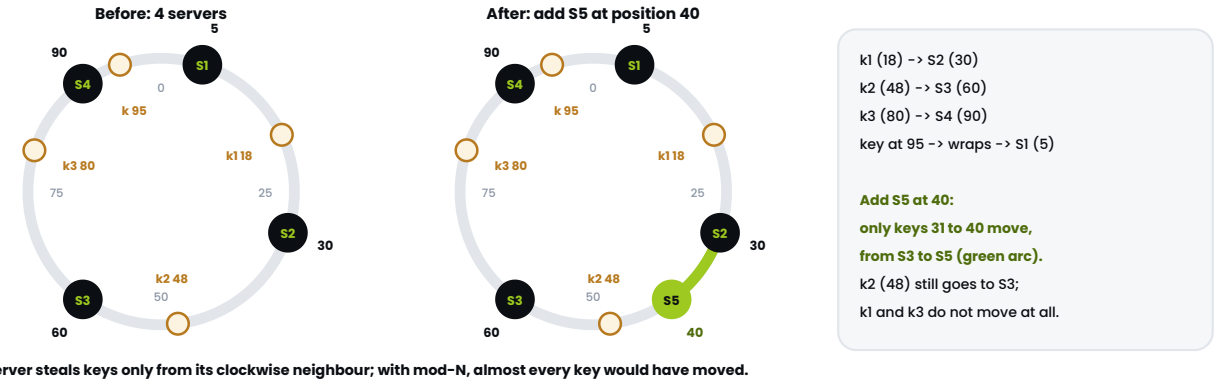
1. Map both servers and keys onto the same circle of hash values (the **ring**).
2. A key belongs to the **first server you meet going clockwise** from the key's position.
3. Adding a server takes over only the keys between it and its predecessor; removing a server hands its keys to the next server. Everything else stays put. The paper calls this **monotonicity**: when buckets are added, items move only to the new buckets [S14].
4. Each real server is placed at many points (**virtual nodes**) so its share of the ring is even [S14].

MEMORY HOOK: CLOCKWISE NEIGHBOUR

A new server **only steals from its clockwise neighbour**. mod-N reshuffles the whole class; the ring just adds one chair.

For sho.rt going from 9 to 10 servers, consistent hashing moves about **1 / 10 = 10%** of keys instead of 90%.

CONSISTENT HASHING: POSITIONS 0 TO 99 ON A CIRCLE; A KEY GOES TO THE FIRST SERVER CLOCKWISE



A new server steals keys only from its clockwise neighbour; with mod-N, almost every key would have moved.

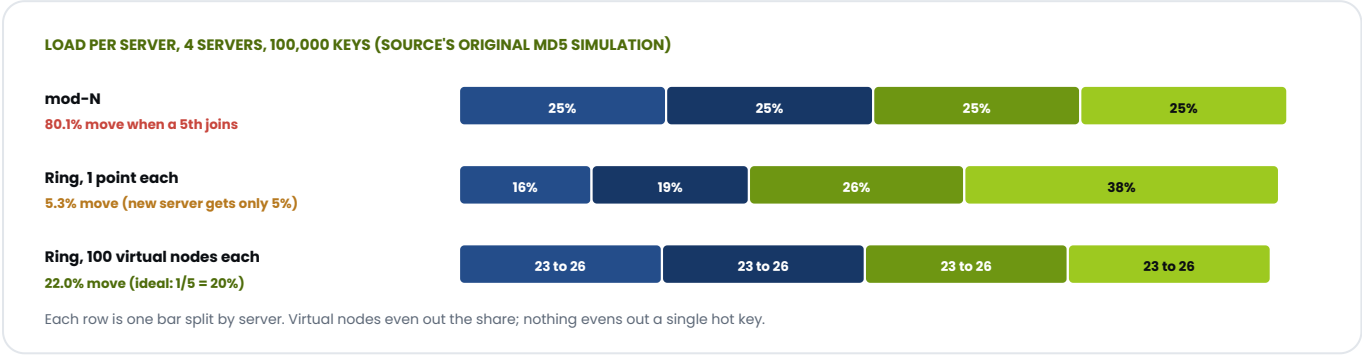
Source diagram 3, redrawn. The ring is drawn as a circle; positions run 0 to 99 and 99 wraps around to 0.

Real configuration. nginx supports `hash $request_uri consistent;`, where the `consistent` parameter switches to **"the ketama consistent hashing method"** [S7]. Google's Maglev network load balancer also uses consistent hashing, together with connection tracking, to limit disruption when machines fail [S17].

Lesson 7 reuses consistent hashing to split a database across machines (sharding). Here it is a **routing** tool.

CONCEPT 5 · VIRTUAL NODES, REAL CONFIGS, WHAT GOES WRONG

Consistent hashing is not magic



SETUP (SOURCE'S SIMULATION, 100,000 KEYS, MD5)	LOAD PER SERVER, 4 SERVERS	KEYS MOVED WHEN A 5TH SERVER JOINS
mod-N	about 25% each	80.1%
Consistent hashing, 1 point per server	16%, 19%, 26%, 38% (very uneven)	5.3% (but the new server got only 5% of keys, so load stayed uneven)
Consistent hashing, 100 virtual nodes per server	23% to 26%	22.0% (the ideal is 1 / 5 = 20%)

WHAT GOES WRONG (ATLASSIAN REPORTEDLY ASKED EXACTLY THIS [S24])

- **Hot keys:** one viral link maps to one server; consistent hashing balances keys, not traffic per key. Fixes: replicate hot keys to several servers, or cache them closer to users (Lesson 6).
- **Too few virtual nodes:** load is lopsided (the 16% to 38% row).
- **Disagreement about membership:** if two load balancers have different server lists, the same key goes to different servers (the paper's "spread" property is about bounding this [S14]).
- **Removal still moves data:** the neighbour of a dead server inherits its keys and its load in one go; with virtual nodes that load spreads across many neighbours.

INTERVIEW MODE: SAY IT LIKE THIS

- "Hash-based routing keeps the same key on the same server, which is useful for caches and session affinity."
- "With mod-N, going from N to N plus 1 servers moves about N over N plus 1 of the keys, 90% when going from 9 to 10."
- "Consistent hashing places servers and keys on a ring, so adding a server moves only about 1 over N of the keys; virtual nodes even out the load."
- "It does not fix hot keys; those need replication or caching."

TINY CHALLENGE

Going from 19 to 20 servers, what fraction of keys move under mod-N, and roughly how many under consistent hashing? Answer: 19 / 20 = 95%; about 1 / 20 = 5%.

MEMORY HOOK: THE TWO FRACTIONS

mod-N moves N/(N+1). The ring moves 1/N. Nine to ten: 90% versus 10%.

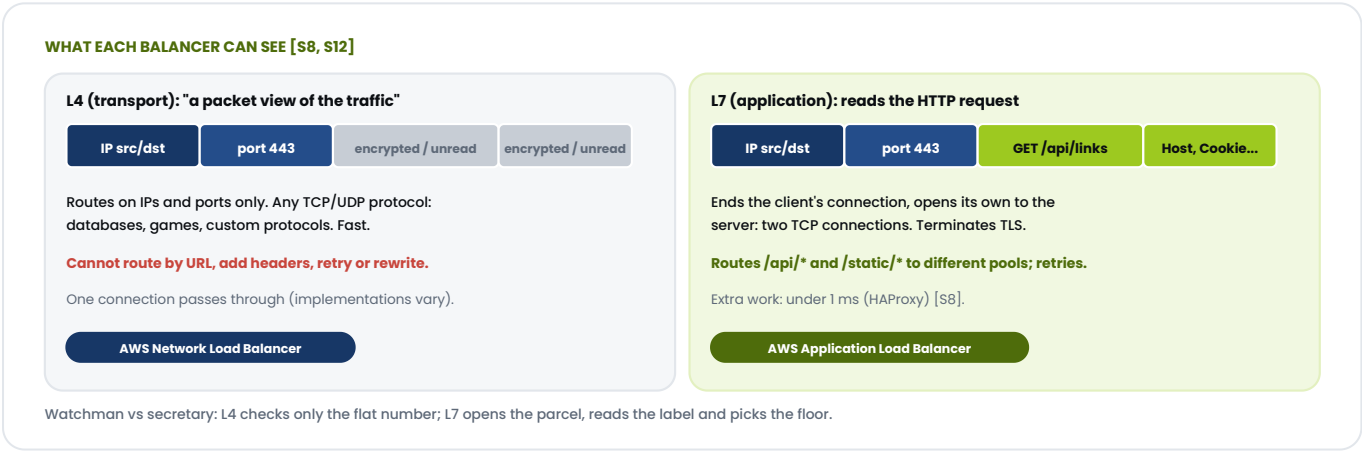
CONCEPT 6 · L4 VS L7 LOAD BALANCING

Look at addresses, or read the request?

"Put a load balancer in front" leaves a choice. The names come from the OSI model's layers.

L4 (transport layer) routes using IP addresses and TCP or UDP ports, without reading application content. HAProxy's FAQ: it has **"a packet view of the traffic"**; it is **"really fast but can't perform any action on the protocol above layer 4"** [S8]. AWS's Network Load Balancer **"operates at the transport layer (layer 4) and supports TCP, TLS, UDP, and QUIC"** [S12] (QUIC is a newer transport built on UDP, used by HTTP/3).

L7 (application layer) reads the request, mainly HTTP: method, path, headers, cookies. It **"acts as a proxy, which means it maintains two TCP connections"**, one with the client and one with the server [S8]. AWS's Application Load Balancer **"operates at the application layer (layer 7) and supports HTTP and HTTPS"** [S12]. HAProxy puts the extra processing at **"less than a millisecond"** [S8].



Watchman vs secretary: L4 checks only the flat number; L7 opens the parcel, reads the label and picks the floor.

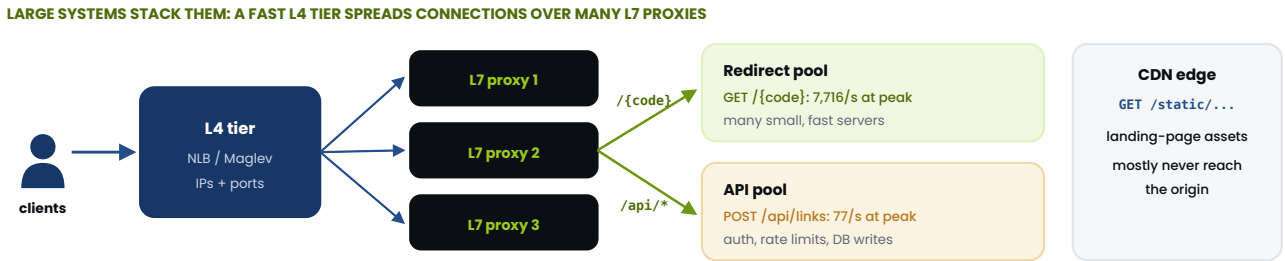
	L4	L7
Sees	Source and destination IP, ports, protocol	All of that plus HTTP method, path, headers, cookies
Connections	Passes the client's connection through (implementations vary)	Ends the client's connection and opens its own to the server [S8]
Route by URL path or header	**No**	**Yes** (/api/* to one pool, /static/* to another)
TLS	Can pass encrypted traffic straight through	Usually terminates TLS so it can read HTTP
Add headers (X-Forwarded-For), retry, rewrite	**No**	**Yes**
Protocols	Any TCP or UDP: databases, games, custom protocols	HTTP, HTTPS, gRPC (over HTTP/2)
Speed and cost per request	Lower	Slightly higher (under 1 ms extra [S8])
AWS example	Network Load Balancer [S12]	Application Load Balancer [S12]

Just for fun. L4 is the society watchman who checks only your flat number; L7 is the society secretary who opens the parcel, reads the label and decides which floor it really belongs to.

CONCEPT 6 · STACKING, AND SHO.RT'S ROUTING

A fast L4 tier in front of a smart L7 tier

They are often stacked. AWS lets a Network Load Balancer send traffic to an Application Load Balancer as its target; the documentation's example notes that such a target group "must be used by a Network Load Balancer" [S13]. The pattern: a fast L4 tier spreads raw connections over many L7 machines, which then make HTTP-aware decisions. Google describes Maglev as its "network load balancer" (an L4 role), which receives packets spread across Maglev machines by routers using **Equal Cost Multipath (ECMP)** [S17].



Only an L7 balancer can split /api/* from /{code}, because only it reads the path. An L4 balancer sends all traffic to one pool.

PATH	VOLUME (ASSUMPTION)	WHAT IT NEEDS
`GET /{code}` redirects	7,716 per second at peak	Many small, fast servers
`POST /api/links` creates	77 per second at peak (Lesson 4)	Authentication, rate limiting, database writes
`GET /static/...` landing-page assets	Served by the CDN (Concept 10)	Nothing at the origin most of the time

Only an L7 balancer can send /api/* to an "API" pool and /{code} to a "redirect" pool, because only it reads the path. An L4 balancer would send all three to the same pool.

A note on terms. "L4" and "L7" are labels from a reference model; real protocols do not fit it perfectly (TLS, for example, sits awkwardly between layers). Interviewers use the labels loosely, as here.

WHAT GOES WRONG

Choosing L4 and later needing path routing, header-based **canary releases** (sending a small slice of users to a new version first) or per-request retries; or choosing L7 for a non-HTTP protocol such as a database connection.

INTERVIEW MODE: SAY IT LIKE THIS

- "An L4 balancer routes on IPs and ports, is protocol-agnostic and fast, but cannot see URLs."
- "An L7 balancer terminates the connection, reads HTTP, and can route by path, header or cookie, terminate TLS, and retry, for under a millisecond of extra work."
- "Large systems often put an L4 tier in front of an L7 tier."

TINY CHALLENGE

You need to load-balance PostgreSQL connections. L4 or L7? Answer: L4: the protocol is not HTTP.

CONCEPT 7 · HEALTH CHECKS

How does the balancer know a server is dead?

A load balancer that keeps sending traffic to a dead server is worse than no load balancer: one in nine users fails, at random.

Active checks

The balancer sends a probe (an HTTP `GET /healthz`, or a TCP connect) every **interval**; a server is marked unhealthy after **unhealthy-threshold** failures in a row and healthy again after **healthy-threshold** successes in a row.

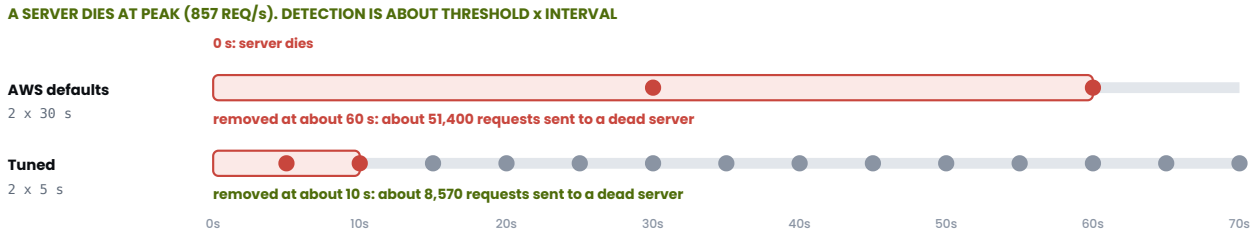
Passive checks

The balancer watches real traffic. nginx's open-source version does this "in-band": if a request to a server fails, nginx marks it failed for a while [S6]. Defaults: `max_fails` is 1 and `fail_timeout` is 10 seconds [S7]. After the timeout, nginx probes the server with live client requests [S6].

AWS target-group health-check defaults. They differ slightly between the Application Load Balancer (ALB, layer 7) and the Network Load Balancer (NLB, layer 4) [S31, S32]:

SETTING	ALB DEFAULT (HTTP, INSTANCE OR IP TARGETS) [S31]	NLB DEFAULT [S32]	NLB DOCUMENTED RANGE [S32]
Check protocol	HTTP	TCP	
Interval	30 seconds	30 seconds	5 to 300 s
Timeout	5 seconds	6 s for HTTP checks; 10 s for TCP and HTTPS checks	2 to 120 s
Unhealthy threshold	2 failures in a row	2	2 to 10
Healthy threshold	5 successes in a row	5	2 to 10
Path	/	(used only for HTTP or HTTPS checks)	

Worked example: how long does a dead server get traffic? One of sho.rt's 9 servers dies at peak. It was receiving 7,716 / 9 = **857 requests per second**.



Red dots = failed probes. "About", because the death can land anywhere in an interval and each probe can wait up to the timeout.

Recovery is slower on purpose: healthy threshold 5 x 30 s = 150 s before a recovered server gets traffic, so a flickering server does not keep rejoining.

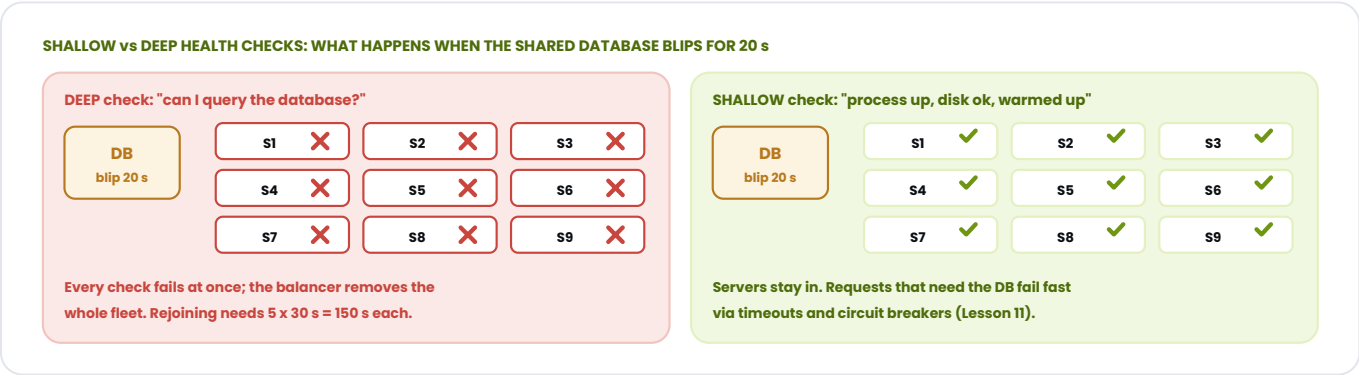
CONCEPT 7 · DETECTION TIME, SHALLOW VS DEEP, FLAPPING

When the health checks are the outage

CONFIGURATION	TIME TO MARK UNHEALTHY (ABOUT THRESHOLD X INTERVAL)	REQUESTS SENT TO THE DEAD SERVER MEANWHILE
AWS defaults: 2 x 30 s	about 60 s	857 x 60 = about 51,400
Tuned: 2 x 5 s	about 10 s	857 x 10 = about 8,570
Plus passive checks (nginx-style, mark failed on first error)	about one failed request per balancer, then 10 s out [S7]	a handful, if the balancer also retries on another server

"About", because the first failed probe can arrive anywhere inside an interval, and each probe can wait up to the timeout. Recovery is slower on purpose: 5 successes x 30 s = **150 s** before a recovered server gets traffic again, so a server that flickers does not keep rejoining.

Shallow vs deep checks. A shallow check ("the process answers") misses a server whose database connection is broken. A deep check ("can I query the database?") catches that, but if the shared database blips, every server fails its check at once and the balancer removes the **entire fleet**. A common middle path: the health endpoint checks only what is **local** to the server (process up, disk not full, warmed up), and dependency failures are handled by timeouts and circuit breakers (Lesson 11).



FLAPPING, A REAL EXAMPLE (AWS, 20 OCTOBER 2025 [S1])

The NLB health-check subsystem brought new machines into service before their network state had fully propagated, so health checks kept **"alternating between failing and healthy"**. Nodes were repeatedly removed and returned, the extra load made the health-check subsystem itself degrade, and for multi-zone load balancers automatic failover took capacity out of service. Connection errors ran from 5:30 AM; engineers stopped them spreading by **disabling automatic health-check failover at 9:36 AM** and re-enabled it at 2:09 PM (Pacific time) after recovery.

The lesson: health checks are a powerful automatic actor, and a design needs a **limit on how much capacity they may remove**. Some balancers stop removing servers once too many look unhealthy; this lesson has not verified a specific product's behaviour, so check your balancer's documentation.

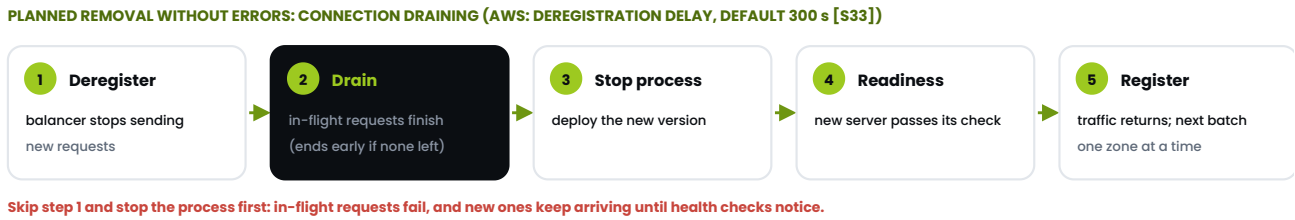
Just for fun. Health checks are like a hostel warden's roll call: useful, but if the warden marks the whole hostel absent because the Wi-Fi went down, nobody gets dinner.

CONCEPT 7 · MID-REQUEST DEATHS, DRAINING, AND THE BALANCER'S OWN SPOF

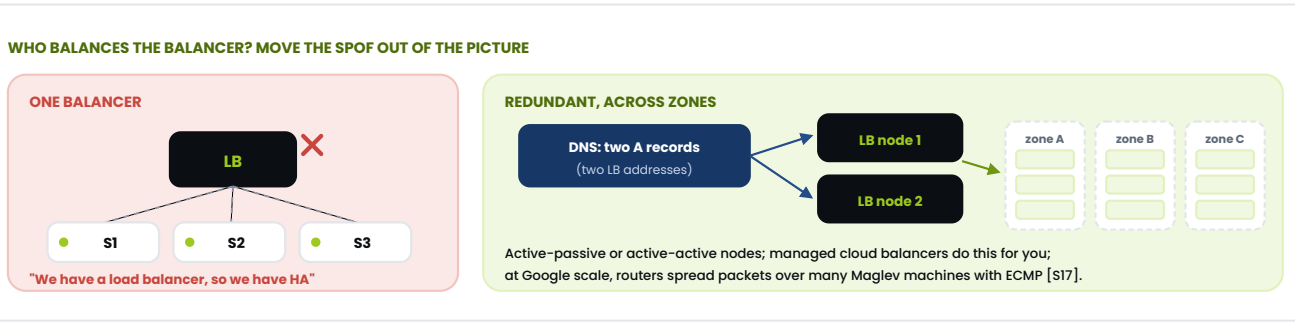
Who balances the balancer?

A server dies mid-request. Requests already on that server fail: the client sees a reset connection or, through a proxy, a **502 or 504** (Lesson 2). An L7 balancer can retry on another server, but only safely for **idempotent** requests (Lesson 2): retrying a `GET /ab12` is fine; blindly retrying a payment POST is not, unless it carries an idempotency key.

Planned removals (deploys) use connection draining: in-flight requests finish first. On AWS, a deregistering target stops receiving new requests, and the balancer waits up to the deregistration delay, **300 seconds by default**, finishing early if the target has no in-flight requests [S33].



Who balances the balancer? The load balancer is now the SPOF from the hall challenge. Answers: run at least two balancer nodes (active-passive or active-active), across zones; managed cloud balancers do this for you; at Google scale, routers spread packets across many Maglev machines with ECMP [S17]; and DNS can return more than one address (Concept 8).



INTERVIEW MODE: SAY IT LIKE THIS

- "Health checks probe each server on an interval and remove it after a threshold of failures; detection time is roughly threshold times interval."
- "With AWS defaults that is about 60 seconds; I would shorten it and add passive checks."
- "I keep health endpoints shallow so a shared dependency's blip does not remove the whole fleet, and I cap how much capacity automation can remove."
- "In-flight requests on a dying server fail; I retry only idempotent requests, and drain connections on deploys."

TINY CHALLENGE

Interval 10 s, unhealthy threshold 3, 1,000 requests per second to the server. About how many hit it after it dies? Answer: 3 x 10 = 30 s; 30,000.

MEMORY HOOK: T X I

Detection = **T**hreshold x **I**nterval. "TI" sounds like "tea": the time the dead server keeps taking orders.

CONCEPT 8 · DNS RESOLUTION

How does sho.rt become an IP address?

Lesson 2 listed "DNS lookup" as step one of a request. This concept opens that box: how a name becomes an address, how long the answer is remembered, and how DNS itself can spread traffic.

With nothing cached, Cloudflare's description [S2]:

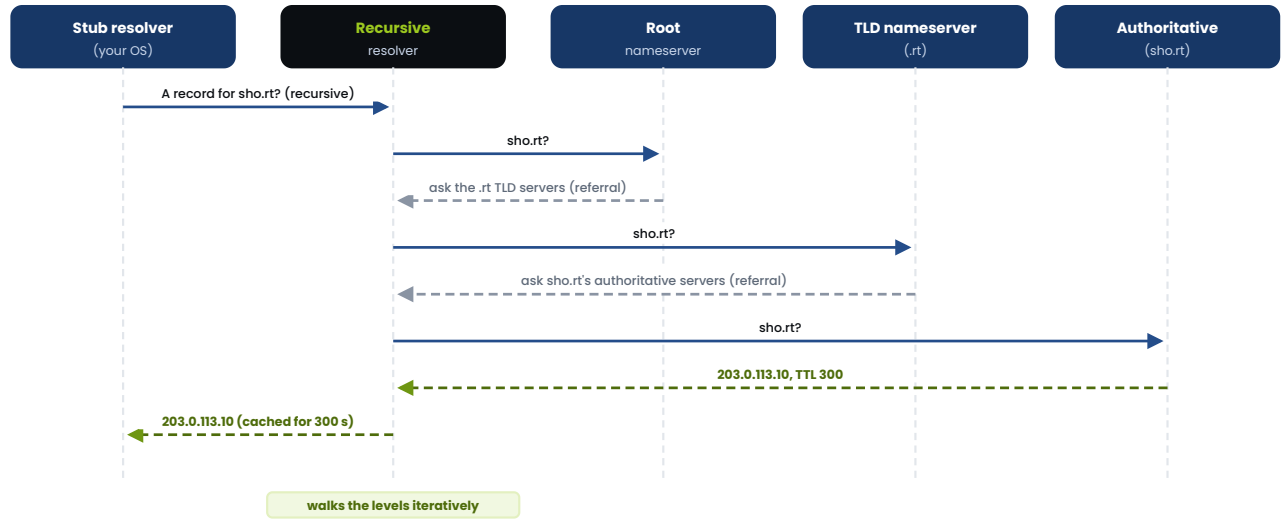
1. Your OS's **stub resolver** sends the query to a **recursive resolver** (usually your ISP's, or a public one such as 1.1.1.1).
2. The resolver asks a **root** nameserver, which refers it to the **TLD** nameserver for .rt.
3. The resolver asks the TLD nameserver, which refers it to the domain's **authoritative** nameserver.
4. The resolver asks the authoritative nameserver, which returns the **A record** (the IP address).
5. The resolver returns the answer to your OS and **caches** it.

RFC 1034 names the two styles: a server answering **iteratively** "refers the client to another server and lets the client pursue the query"; a server answering **recursively** "returns either an error or the answer, but never referrals" [S3]. Your stub resolver asks recursively; the recursive resolver then walks root, TLD and authoritative iteratively.

MEMORY HOOK: STUDENTS READ ROUGHLY TWENTY ARTICLES

Stub, **R**ecursive, **R**oot, **T**LD, **A**uthoritative.

DNS RESOLUTION WITH NOTHING CACHED [S2, S3]



The stub asks once and waits (recursive); the resolver follows referrals (iterative). After the first lookup, its cache answers everyone for the TTL.

Source diagram 2, redrawn.

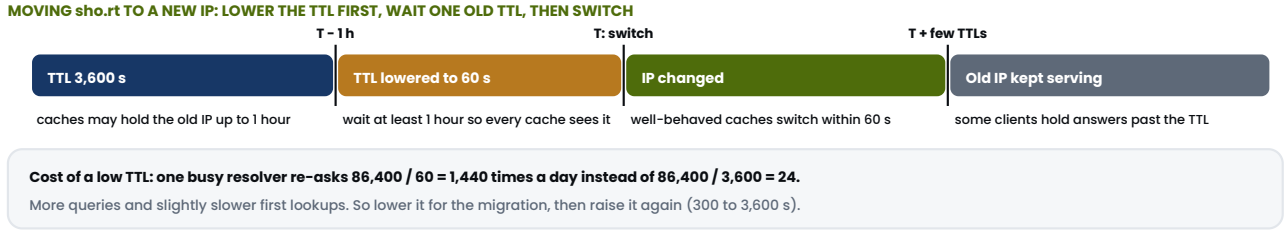
CONCEPT 8 · FACTS WORTH SAYING CORRECTLY, AND TTL

TTL: the dial that controls change

Facts worth saying correctly. There are **13 root nameserver identities**, but each is many machines worldwide reached by anycast; Cloudflare's page says "over 600" instances [S2]. That figure is a floor, and it may not be current: the live count is published at root-servers.org, so check there before quoting a number. A **CNAME** "identifies its owner name as an alias", and the resolver restarts the lookup at the real name [S3]. In practice, the browser and OS also cache answers, so most lookups never leave your machine or your ISP.

TTL. "The TTL describes how long a RR [resource record] can be cached before it should be discarded" [S3]. It is in seconds.

Worked example: moving sho.rt to a new load balancer IP.



STEP	CALCULATION	RESULT
Current TTL	3,600 s	Caches may keep the old IP for up to 1 hour after you change it
Plan: lower TTL to 60 s first	Wait at least the old TTL (1 hour) so every cache has picked up the 60 s value	Then change the IP
After the change	Well-behaved caches switch within 60 s	Keep the old IP serving for a while anyway: some clients and resolvers hold answers longer than the TTL (how common this is is uncertain; measure traffic on the old IP)
Cost of a low TTL	One busy resolver re-asks 86,400 / 60 = 1,440 times a day instead of 86,400 / 3,600 = 24	More queries and slightly slower first lookups

Just for fun. TTL is the expiry date on a packet of milk: shops may keep selling the old packet until that date, no matter how many times you tell them a fresh batch has arrived.

CONCEPT 8 · DNS-BASED LOAD BALANCING AND DIG

DNS is the coarse layer

DNS-based load balancing. A name can have several A records; rotating them is **round-robin DNS** [S11]. Cloudflare's algorithm guide even describes round robin as "distributes traffic to a list of servers in rotation using the Domain Name System" [S10]. Limits of plain DNS balancing: answers are **cached**, so you cannot shift traffic faster than the TTL; the **client chooses** among returned addresses; and basic DNS has **no idea whether a server is healthy** unless the DNS provider runs health checks (Cloudflare's product does [S11]). So DNS is usually the coarse layer (which region, which load balancer), and the load balancer is the fine layer (which server).

Commands you can run.

```
dig sho.rt A            # ask your resolver for the A record; the answer shows the remaining TTL
dig +short sho.rt       # just the IP address(es)
dig +trace sho.rt       # walk root -> TLD -> authoritative yourself, like a recursive resolver
dig @1.1.1.1 sho.rt     # ask a specific public resolver
```

Illustrative output for `dig sho.rt A` (**invented**, not captured from a real run: sho.rt is our example name, .rt is not a real TLD, and 203.0.113.0/24 is an address block reserved for documentation):

```
;; ANSWER SECTION:
sho.rt.         287     IN      A       203.0.113.10
sho.rt.         287     IN      A       203.0.113.11
```

Read it as: two A records (two load balancer addresses), and this resolver's cached copy has **287 seconds** of TTL left. Run dig twice a few seconds apart and the number counts down.

WHAT GOES WRONG

The AWS incident in the hook: the record was emptied, and clients failed until it was fixed and cached answers expired [S1]. Also: forgetting to lower the TTL before a migration, assuming a DNS change is instant, or relying on DNS round robin to avoid a dead server.

INTERVIEW MODE: SAY IT LIKE THIS

- "The OS stub resolver asks a recursive resolver, which walks root, TLD and the authoritative server, then caches the answer for its TTL."
- "TTL trades agility for load: a low TTL lets me move traffic quickly but increases queries, so I lower it before a planned change."
- "DNS can spread traffic across regions or load balancers, but it is cached and health-blind by default, so fine-grained balancing and failover belong in the load balancer."

TINY CHALLENGE

A record has TTL 300 s. You change the IP at 10:00:00. By when should well-behaved caches have the new IP?
Answer: By about 10:05:00.

CONCEPT 9 · FORWARD PROXY VS REVERSE PROXY

Ask one question: whom does it act for?

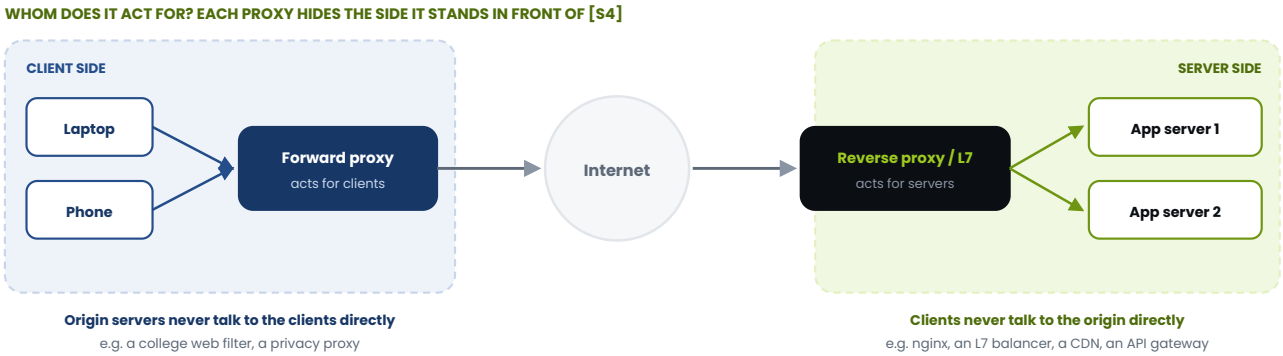
"Proxy" is used for two opposite things. Mixing them up is a classic MCQ trap.

Forward proxy [S4]

"A server that sits in front of a group of client machines". It makes requests to websites on the clients' behalf, so "no origin server ever communicates directly with that specific client". Uses: an office or college proxy that filters sites, enforcing browsing rules, hiding the user's identity.

Reverse proxy [S4]

"A server that sits in front of web servers". It receives clients' requests for the servers, so "no client ever communicates directly with that origin server". Benefits: load balancing, protection (the origin's IP stays hidden, which helps against attacks), caching, and TLS work offloaded from the origin.



Source diagram 4, redrawn. A forward proxy is configured by the clients (or their network); a reverse proxy by the site owner, and clients usually do not know it exists.

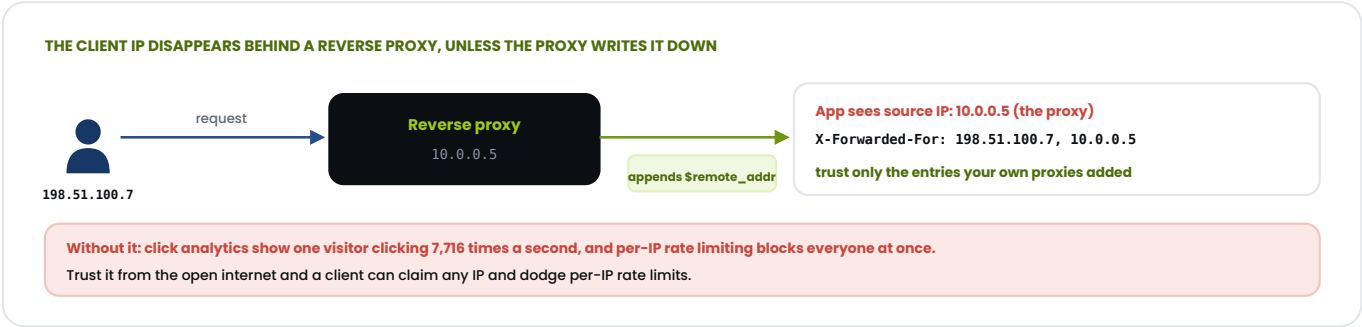
	FORWARD PROXY	REVERSE PROXY
Sits in front of	Clients	Servers
Who sets it up	The client side (a company, a college, a user)	The website owner
Hides	The client from the server	The server from the client
Examples	Office web filter, a privacy proxy	nginx in front of an app, an L7 load balancer, a CDN, an API gateway

Where the API gateway fits. An API gateway is a reverse proxy specialised for APIs: it routes /users/* and /links/* to different services, checks authentication, applies rate limits (Lesson 1), and records metrics. A **load balancer** spreads load across copies of the **same** service; a **gateway** routes across **different** services and applies policies. In practice one product often does both, and an L7 load balancer is itself a reverse proxy.

CONCEPT 9 · THE CLIENT IP DISAPPEARS

Behind a proxy, every user looks like 10.0.0.5

Worked example. Behind a reverse proxy, sho.rt's app sees every request coming from the proxy's IP, say 10.0.0.5. The click-analytics table now says one visitor clicked 7,716 times a second. The fix: the proxy adds an **X-Forwarded-For** header with the client's address, and the app reads that header, but only trusts it when the request really came from your proxy (anyone can send a fake header).




```
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

The variable `$proxy_add_x_forwarded_for` is the incoming X-Forwarded-For header with the client's address (`$remote_addr`) appended after a comma, or just the client's address if the header was absent [S34]. That "appended" detail is why the header can hold a chain such as `198.51.100.7, 10.0.0.5`, and why only the entries added by your own proxies can be trusted.

WHAT GOES WRONG

Trusting X-Forwarded-For from the open internet (a client can claim any IP and dodge per-IP rate limits); and a reverse proxy whose timeout is shorter than the backend's slow requests, producing 504s (Lesson 2, Scenario 4).

Just for fun. A forward proxy is the friend who orders food for the whole hostel so the restaurant never learns who ate what. A reverse proxy is the restaurant's single delivery window, so you never see which cook made your biryani.

INTERVIEW MODE: SAY IT LIKE THIS

- "A forward proxy acts for clients and hides them from servers; a reverse proxy acts for servers and hides them from clients."
- "Load balancers, CDNs and API gateways are all reverse proxies with different jobs."
- "Behind a reverse proxy I read the client IP from X-Forwarded-For, trusting it only from my own proxies."

TINY CHALLENGE

A VPN-style service your laptop uses to browse: forward or reverse? Answer: Forward: it acts for you, the client.

MEMORY HOOK: FORWARD = FOR THE USER

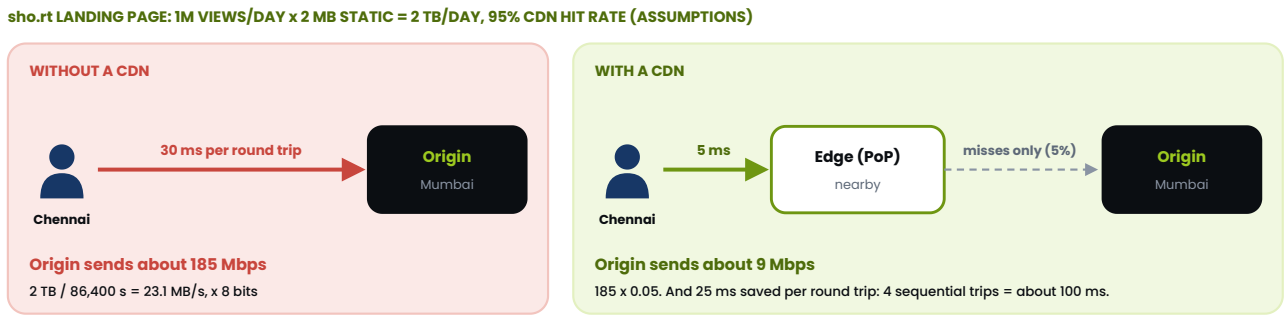
Forward proxy works **for** the people sending requests. **Reverse** proxy works for the people receiving them.

CONCEPT 10 · CDN BASICS

Keep copies near the users

Some content is the same for everyone (images, JavaScript, CSS, video segments) and users are far away. Serving it from one origin costs bandwidth and round trips.

A CDN runs many edge servers near users. Cloudflare's description: an edge server keeps content near requesting clients; on a cache miss it fetches the file from the origin, delivers it, and caches it for later requests [S5]. Two wins: **lower latency** (shorter distance) and **lower origin load** (edge servers "take the load off of one or more origin servers", including during traffic spikes) [S5]. What stays at the origin: server-side code and data, such as databases [S5].



The edge keeps copies of unchanging content. Personal, live and write traffic still goes to the origin, like a kirana store that will not bake your custom cake.

LINE (ALL ASSUMPTIONS)	CALCULATION	RESULT
Daily static bytes	1,000,000 x 2 MB	2 TB a day
Average bandwidth without a CDN	2 TB / 86,400 s = 23.1 MB/s; x 8	about 185 Mbps from the origin
Origin bandwidth with a CDN (5% misses)	185 x 0.05	about 9 Mbps
Latency for a user in Chennai (assumed RTTs)	origin in Mumbai: 30 ms per round trip; nearby edge: 5 ms	25 ms saved on every round trip; 4 sequential round trips save about 100 ms

What about the redirects themselves? Lesson 4 found redirects need only about 31 Mbps at peak, so a CDN is not needed for bandwidth. A CDN could cache 301 redirects at the edge for latency, but then clicks never reach the origin and **click counting breaks**. That is a product decision, and how long edges may keep a copy is a caching question: Lesson 6.

CONCEPT 10 · A REAL SYSTEM, AND WHAT GOES WRONG

More boxes in the path are not automatically faster

A REAL SYSTEM: UBER'S 2025 EDGE REDESIGN [S19]

Uber put Google's Global External Application Load Balancer in front of its own data centres, with Cloud CDN for caching and Cloud Armor for DDoS (distributed denial-of-service) protection, and reported a **10% p99 latency improvement** for mobile users. Uber also found that running its own Envoy proxies in many regions **"did not consistently improve latency for all users"**, because it added hops.

Just for fun. A CDN is your neighbourhood kirana store stocking the popular biscuits so you do not take a bus to the factory; it will not, however, bake you a custom cake.

WHAT GOES WRONG

Users see an **old JavaScript file** after a deploy because edges still hold the previous copy (fix: put a version or content hash in the file name, or purge); **personal pages accidentally cached** and shown to other users; and a **CDN outage** taking down sites that have no fallback.

INTERVIEW MODE: SAY IT LIKE THIS

- "A CDN caches static content on edge servers near users; misses go to the origin."
- "It cuts latency and origin bandwidth; with a 95% hit rate the origin serves a twentieth of the static bytes."
- "Dynamic, personal and write traffic still goes to the origin; cache lifetimes and invalidation are the hard part."

TINY CHALLENGE

500 Mbps of static traffic and a 90% hit rate: what does the origin serve? Answer: 50 Mbps.

MEMORY HOOK: HIT RATE IN YOUR HEAD

Origin load = total x **(1 - hit rate)**. 95% hits = one twentieth; 90% = one tenth.

PUT ON A CDN	KEEP AT THE ORIGIN
Images, JavaScript, CSS, video segments, landing-page assets	Logins, dashboards, anything personal
Files whose name changes when content changes (versioned)	Writes (POST, PUT, DELETE) and live data
Content that is the same for everyone	Redirects, if every click must be counted (sho.rt's product decision)

ONE WHITEBOARD, EVERY NUMBER

The routing decisions for sho.rt

The interviewer says: "You sized sho.rt in the estimation step. Now, how does a click in Chennai reach one of your servers, and what happens when things break?"

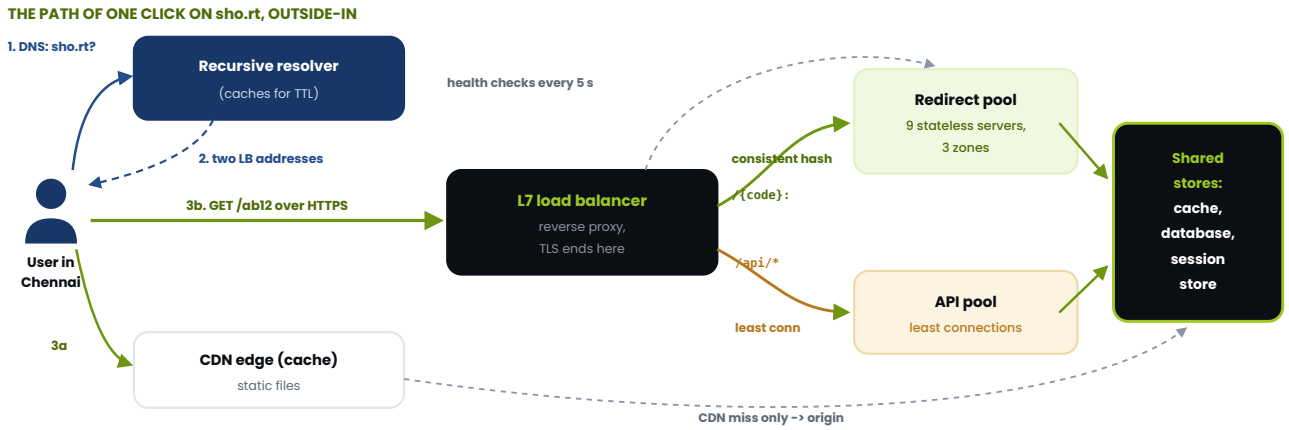
ASSUMPTION (SAY ALOUD)	VALUE
Peak redirects	7,716 per second
Peak creates	77 per second
One app server handles	2,000 simple requests/s
Request time	20 ms
Hot set for caching	33 GB
Zones (new)	3 AZs, 3 redirect servers in each
Health check (new)	interval 5 s, unhealthy 2, healthy 3
DNS TTL (new)	300 s normally; 60 s during migrations
Landing-page static (new)	2 TB a day, 95% CDN hit rate

Values from Lesson 4 unless marked "new".

#	DECISION	NUMBERS BEHIND IT
1	DNS: two A records pointing at two load balancer addresses, TTL 300 s	A failed balancer address can be swapped within about 300 s; resolvers re-ask 86,400 / 300 = 288 times a day each
2	CDN for static assets only	Origin static bandwidth drops from about 185 Mbps to about 9 Mbps; redirects stay at the origin so every click is counted
3	L7 load balancer (a reverse proxy) terminating TLS	Routes /api/* (77 writes/s, with authentication) and /{code} (7,716/s) to different pools; costs under 1 ms [S8]
4	Horizontal redirect pool: 9 stateless servers across 3 zones	7,716 / 9 = 857/s each (43%); lose one zone: 7,716 / 6 = 1,286 (64%), still under 2,000
5	Consistent hashing on the short code (redirect pool)	Each local cache holds about 33 / 9 = 3.7 GB instead of 33 GB; a 10th server moves about 10% of keys, not 90%
6	Least connections for the API pool	Creates vary in cost (database writes), so in-flight count beats turns
7	Health checks: active every 5 s, threshold 2, plus passive	A dead redirect server gets about 857 x 10 = 8,570 requests before removal (vs about 51,400 with 2 x 30 s); idempotent GETs are retried elsewhere
8	Health endpoint is shallow	A brief database blip must not remove all 9 servers at once
9	Stateless: dashboard logins use a shared session store	With in-memory sessions, 8 in 9 (89%) of follow-ups would look logged out
10	No single balancer: the managed balancer runs in all 3 zones	The balancer is no longer the SPOF

DRAW IT OUTSIDE-IN

The path of one click



Only the shared stores hold state; every box to their left can be duplicated, which is what makes horizontal scaling possible.

Source diagram 1, redrawn. Notice that only the shared stores hold state; every box to their left can be duplicated.

THE 60-SECOND SPOKEN VERSION

"DNS returns two balancer addresses with a 300-second TTL. Static assets come from a CDN, which cuts origin bandwidth from about 185 to about 9 megabits per second. An L7 balancer terminates TLS and routes /api to a least-connections pool and redirects to a pool of 9 stateless servers across 3 zones, hashed by short code with consistent hashing so each server caches about 3.7 gigabytes. Losing a zone leaves us at 64% load. Health checks every 5 seconds remove a dead server in about 10 seconds, and redirects are retried because they are idempotent. Hot links get replicated to every cache."

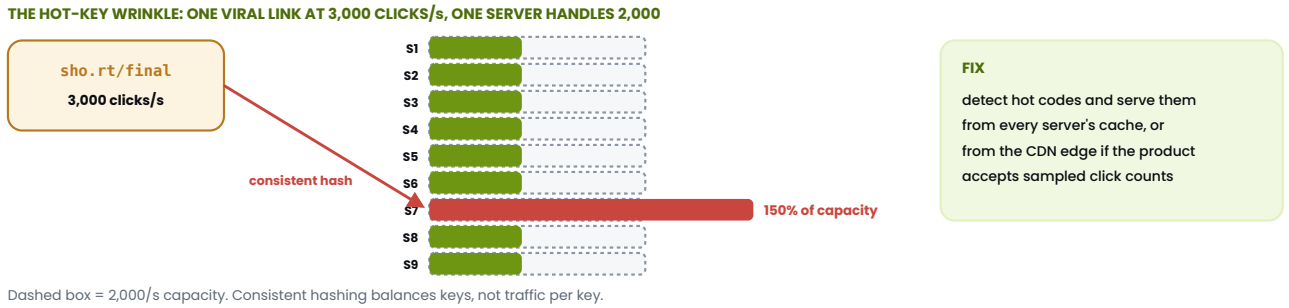
INTERVIEW MODE: SAY IT LIKE THIS

- "I go outside-in: DNS, CDN, load balancer, servers. For each I say the choice, the number that justifies it, and what happens when it fails: zone loss leaves 64% load, a dead server is removed in about 10 seconds, and hot keys are replicated."

THE FOLLOW-UP YOU SHOULD EXPECT

The hot-key wrinkle

Suppose one link goes viral at **3,000 clicks per second**. Consistent hashing sends all of them to one server, which handles 2,000. The fix is to detect hot codes and serve them from every server's cache, or from the CDN edge if the product accepts sampled click counts. This is exactly the "issues with consistent hashing" follow-up.



Just for fun. Every Indian startup's dream: a cricket-final meme link. Every on-call engineer's nightmare: the same link.

INTERVIEW MODE: SAY IT LIKE THIS

- "I draw the path outside-in. Only the shared stores hold state, so every box before them can be duplicated. For DNS I name stub, recursive, root, TLD and authoritative. For consistent hashing I show that a new server takes keys only from its neighbour on the ring."

THINGS THAT SOUND RIGHT AND ARE NOT

Common misconceptions

BELIEF	CORRECTION
"Horizontal scaling is always better."	It needs stateless servers and a balancer. For a component that fits one machine, or a single database primary, vertical scaling is often the simpler right answer.
"Round robin balances load."	It balances request counts. If requests or servers differ, use weights or least connections.
"Consistent hashing gives perfectly even load."	Without virtual nodes it can be badly uneven (16% to 38% in the 4-server simulation), and it never fixes a single hot key.
"An L7 balancer is slow."	HAProxy puts the extra work at under a millisecond [S8]; choose by what you need to see, not by fear of overhead.
"Health checks make failures invisible."	A dead server still receives traffic until the threshold is hit: about 60 s with AWS defaults [S31, S32]; and over-eager checks can remove healthy capacity [S1].
"DNS changes are instant."	Answers are cached for the TTL [S3]; lower it before a planned change and keep the old address serving for a while.
"There are 13 root DNS servers."	There are 13 root server identities, each served by many machines worldwide via anycast (Cloudflare: "over 600" in total [S2]).
"A proxy is a proxy."	Forward proxies act for clients; reverse proxies act for servers [S4]. Load balancers, CDNs and API gateways are reverse proxies.
"Put everything on the CDN."	CDNs serve cacheable content; personal, dynamic and write traffic goes to the origin [S5].

Just for fun. Bonus misconception: "We have a load balancer, so we have high availability." Congratulations, you have moved the single point of failure one box to the left.

INTERVIEW MODE: SAY IT LIKE THIS

- "Horizontal scaling needs statelessness; round robin balances counts not work; consistent hashing needs virtual nodes and still has hot keys; health checks have a detection delay; DNS is cached for its TTL; and forward and reverse proxies act for opposite sides."

TEAR OUT THIS PAGE

One-page cheat sheet

WHAT THEY ASK	THE ANSWER	THE TRAP
"Vertical vs horizontal?"	Bigger machine vs more machines; vertical has a ceiling and is a SPOF; horizontal needs stateless servers and a balancer	Saying horizontal is free
"What is a stateless server?"	Keeps no per-user state between requests; state lives in a shared store or a signed token [S15]	Forgetting local disk, in-memory counters, cron jobs
"Sticky sessions?"	Same user to the same server; uneven load, sessions die with the server	Recommending them as the main design
"Round robin vs least connections?"	Turns vs fewest in-flight; least connections adapts to slow servers and long connections [S6, S9]	Least connections without health checks (fast-failing server looks idle)
"Weighted round robin share?"	Weight / sum of weights; 3 : 1 : 1 gives 60/20/20 [S6]	Assuming weights are percentages
"Why consistent hashing?"	Adding a server moves about 1/N of keys; mod-N moves about N/(N+1)	Forgetting virtual nodes and hot keys
"L4 vs L7?"	IPs and ports vs HTTP-aware proxy with two connections [S8]; NLB vs ALB [S12]	Saying L4 can route by URL
"How do health checks work?"	Probe every interval; unhealthy after N failures; detection = about threshold x interval	Deep checks that fail the whole fleet on one dependency blip
"Server dies mid-request?"	In-flight requests fail; retry only idempotent ones; drain on deploys	Blindly retrying payments
"Who balances the balancer?"	Redundant balancer nodes across zones; multiple DNS addresses; ECMP at large scale [S17]	Leaving the LB as a SPOF
"Walk through DNS."	Stub -> recursive resolver -> root -> TLD -> authoritative -> cached for TTL [S2, S3]	Saying the browser asks the root directly
"What is TTL?"	Seconds a DNS answer may be cached [S3]; lower it before migrations	Expecting instant propagation
"Forward vs reverse proxy?"	Acts for clients vs acts for servers [S4]	Calling a CDN a forward proxy
"LB vs API gateway?"	Copies of one service vs routing plus policy across services	Insisting they never overlap
"What is a CDN?"	Edge caches near users; misses go to origin [S5]	Caching personal pages

WHAT YOU WILL ACTUALLY CONFIGURE AT WORK

Where this shows up in real systems

An nginx reverse proxy and load balancer. A minimal config that combines this lesson (directive names from the nginx documentation [S6, S7]; tune values for your system):

```
upstream redirect_pool {
    hash $request_uri consistent;           # ketama consistent hashing [S7]
    server 10.0.1.11 max_fails=1 fail_timeout=10s;   # passive checks; these are the defaults [S7]
    server 10.0.1.12;
    server 10.0.1.13;
}
upstream api_pool {
    least_conn;                             # fewest active connections [S6]
    server 10.0.2.21 weight=2;              # twice the share of a weight-1 server
    server 10.0.2.22;
}
server {
    listen 80;
    location /api/ { proxy_pass http://api_pool; }
    location /     { proxy_pass http://redirect_pool; }
}
```

Read it as: an L7 reverse proxy routing by path to two pools with two different algorithms. Open-source nginx does passive in-band checks [S6]; nginx's documentation says periodic (active) health checks are "available as part of our commercial subscription" [S7].

AWS load balancers and target groups. AWS offers an Application Load Balancer (layer 7: HTTP, HTTPS), a Network Load Balancer (layer 4: TCP, TLS, UDP, QUIC) and a Gateway Load Balancer (layer 3) [S12]. Health checks are set on the target group: interval (default 30 s), timeout (default 5 s for an ALB HTTP check; 6 s for an NLB HTTP check, 10 s for NLB TCP and HTTPS checks), healthy threshold (default 5), unhealthy threshold (default 2), path (default /) [S31, S32]. Interval and unhealthy threshold are exactly the detection-time arithmetic from Concept 7. The target-group attribute `deregistration_delay.timeout_seconds` controls draining; its default is 300 s [S33].

dig and curl on your laptop. `dig +trace` and `dig` (Concept 8) show the resolution chain and the remaining TTL. `curl -sI https://example.com` prints only response headers; behind a CDN you will often see cache-status headers, but their names differ by provider, so check your CDN's documentation rather than memorising one.

AT VERY LARGE SCALE

Maglev, Uber, and the numbers they report

Google Maglev (layer 4, very large scale)

Maglev is "Google's network load balancer", runs on ordinary Linux servers, receives packets spread by routers with ECMP, uses consistent hashing and connection tracking, and "has been serving Google's traffic since 2008"; one machine can "saturate a 10Gbps link with small packets" [S17].

Uber (a): service-to-service

Uber's on-host proxies used "least-pending request" balancing. With thousands of backend tasks per service, full connection meshes become huge (two services with 10,000 tasks each would need 100,000,000 TCP connections), so each proxy talks only to a subset of backends [S18] (May 2022).

Uber (b): the edge

In 2025 Uber replaced Envoy proxy VMs in 16 cloud regions with a global L7 load balancer plus CDN and DDoS protection, serving over 1 million HTTP requests per second, and reported 2.6% better p50 and 10% better p99 latency for mobile users [S19].

CONCEPT IN THIS LESSON	WHAT YOU WILL SEE IN PRODUCTION
Algorithm	`least_conn`, `hash ... consistent`, `weight=` in nginx; "least-pending requests" at Uber [S18]
Health checks	Target-group interval, timeout and thresholds on AWS [S31, S32]; max_fails and fail_timeout in nginx [S7]
L4 vs L7	Network Load Balancer vs Application Load Balancer [S12]; Maglev as an L4 tier [S17]
Draining	deregistration_delay.timeout_seconds, default 300 s [S33]
DNS and TTL	dig output: the remaining TTL counting down
CDN	Cache-status response headers (names differ by provider)

INTERVIEW MODE: SAY IT LIKE THIS

- "In practice this is an L7 proxy such as nginx or a cloud Application Load Balancer, configured with pools, an algorithm and health checks; an L4 tier such as a Network Load Balancer or Maglev in front at large scale; DNS with sensible TTLs; and a CDN for static content."

ONE NUMBER FROM EACH COMPANY, WORTH REMEMBERING

Maglev: **since 2008**, a 10 Gbps link per machine. Uber edge: **over 1 million** HTTP requests a second, **10% better p99**. Uber mesh: 10,000 x 10,000 tasks would be **100 million** connections, so each proxy talks to a subset.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 1 and 2

Two questions are Source-confirmed, several are Candidate-reported, and the rest are Pattern-only. Labels are kept exactly as the evidence allows (see the honest note on page 2).

Q1 "What is a load balancer?" SOURCE-CONFIRMED

Amazon, Software Engineer, Chennai; interview April 2025 (review posted 20 Feb 2026). The listed question is "what is a load balancer"; the same loop's design round was an online retail system that handles peak sales [S20].

Model answer. "A load balancer gives clients one address and spreads their requests across several servers. It picks a server with an algorithm, such as round robin or least connections, and runs health checks so it only sends traffic to healthy servers. That gives capacity, because I can add servers, and availability, because one server failing removes only a slice."

> **Why not just buy a bigger server?** "A bigger server has a ceiling and is still a single point of failure. With 7,716 requests per second at peak and 2,000 per server, 9 small servers across zones survive losing a whole zone at 64% load."

> **A server dies. How many requests fail?** "Until health checks notice: with a 30-second interval and threshold 2, about 60 seconds; at 857 requests per second to that server, about 51,000. With 5-second checks about 8,600, and retries on idempotent requests hide most of those."

> **Isn't the load balancer now the single point of failure?** "Yes, unless it is redundant: two or more balancer nodes across zones, a managed balancer that does this, and DNS returning multiple balancer addresses."

Weak: "It balances the load." (A definition that uses its own name is the interview equivalent of "water is wet water".)

Strong: "One address, an algorithm, health checks, and here is how it fails and how I protect it."

Q2 Types of load balancers (L4 vs L7) and balancing algorithms CANDIDATE-REPORTED

Adobe, online-assessment MCQs: an aggregator says the CS-fundamentals MCQs include "DNS and load balancer types" (page titled "Adobe's Interview Process (2026)") [S22]. Microsoft SDE-2: a 2025 compilation lists "How many types of load-balancers you have used" inside a URL-shortener HLD question [S25].

Model answer. "By layer: an L4 balancer routes on IP addresses and ports, so it is fast and works for any TCP or UDP protocol but cannot see URLs; an L7 balancer is an HTTP-aware proxy that terminates the connection and can route by path, header or cookie, terminate TLS and retry. On AWS that is the Network versus Application Load Balancer. By algorithm: static ones such as round robin, weighted round robin and IP hash, and dynamic ones such as least connections, which look at server state."

> **Why can L7 route by URL and L4 not?** "L7 reassembles the request and reads HTTP, keeping one connection to the client and one to the server; L4 only sees packets and ports."

> **Weights 3, 1, 1. What share does each get?** "Weight over total: 3/5, 1/5, 1/5, so 60%, 20%, 20%."

> **When would you put L4 in front of L7?** "At large scale: a fast L4 tier spreads connections over many L7 proxies, which make the HTTP decisions. AWS supports a Network Load Balancer targeting an Application Load Balancer."

Weak: "Hardware and software load balancers."

Strong: "Layer, algorithm, a product example of each, and when to stack them."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 3 and 4

Q3 "Explain DNS" / "How does DNS resolution work end to end?" SOURCE-CONFIRMED CANDIDATE-REPORTED

Google, Customer Solutions Engineer, Singapore, review dated 8 Jan 2026: "Explain DNS", plus "what happens after you type google.com" [S21] (a solutions-engineering role, not software engineering). Adobe: the aggregator advises reviewing "how DNS resolution works end to end" [S22]. Zomato SDE (post titled "Off-Campus | 2025"): round 1 included what happens when a user enters "zomato.com", from DNS resolution to rendering [S26]. Source-confirmed for Google; Candidate-reported for Adobe and Zomato (the Zomato post shows no date; "2025" may be the graduating batch).

Model answer. "DNS turns a name into an IP address. The browser and OS check their caches; on a miss the OS stub resolver asks a recursive resolver, usually the ISP's. If that resolver has no cached answer, it asks a root server, which refers it to the TLD server for .com; the TLD server refers it to the domain's authoritative nameserver, which returns the A record. The resolver caches it for the record's TTL and returns it. Lesson 2's full URL journey then continues with TCP, TLS and HTTP."

> **Why so many levels?** "Delegation: the root knows only TLDs, each TLD knows only its domains' nameservers, and each domain owner controls its own records. Caching at every level keeps the root from being asked for every lookup."

> **You changed the IP; some users still hit the old server an hour later. Why?** "Their resolvers cached the old answer for the TTL. With a TTL of 3,600 seconds that is up to an hour; lower the TTL ahead of a change, and keep the old IP serving."

> **Can DNS load-balance?** "Yes, with multiple A records or geography-aware answers, for coarse balancing across regions or load balancers. It is cached and health-blind by default, so per-server balancing and fast failover belong in the load balancer."

Weak: "DNS is the phone book of the internet." (A fine opener, but stopping there shows nothing.)

Strong: "Stub, recursive, root, TLD, authoritative, TTL, and what goes wrong."

Q4 Vertical vs horizontal scaling PATTERN-ONLY

None found as a verbatim question. Context (Candidate-reported): an Amazon SDE-2 candidate (Chennai, interview April 2024, older than 24 months) was asked "How delivery systems like swiggy handles so much load?" in a design round [S28]; a Flipkart SDE-2 hiring-manager round included scenario questions on pod scaling and distributing high traffic across instances (page gives months, not the year) [S27].

Model answer. "Vertical scaling makes one machine bigger: simple, no code change, but it has a ceiling, gets expensive at the top end, often needs a restart, and stays a single point of failure. Horizontal scaling adds machines behind a load balancer: it grows further and survives failures, but servers must be stateless and you now run a balancer and health checks. I scale stateless app servers horizontally and often scale a database primary vertically until I must shard."

> **Why does horizontal need statelessness?** "Because the balancer may send each request to a different server; if one server holds the session, the others cannot serve that user."

> **Each server is up 99.9%. One server versus two, either enough?** "One is down about 8.8 hours a year; two independent ones are both down 0.001 squared of the time, about 32 seconds a year. Independence is the catch: same zone or same bad deploy breaks it."

> **Traffic is 10x during a sale. What do you do?** "Autoscale the stateless tier from metrics, pre-warm before the known start time, put static content on a CDN, and check that the database and caches are not the next bottleneck."

Weak: "Horizontal is better."

Strong: "Both, with the condition, the cost, and a number."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 5 and 6

Q5 What is a stateless server? What are sticky sessions and when are they a problem?

PATTERN-ONLY

None found.

Model answer. "A stateless server keeps no per-user data between requests; sessions live in a shared store or in a signed token. Sticky sessions instead pin a user to one server. They are a quick fix but unbalance load, lose sessions when that server dies, and make scaling in and out harder."

> **Why does Twelve-Factor ban sticky sessions?** "Because a process must be disposable and interchangeable; if state lives in one process, killing or adding processes breaks users."

> **IP-based stickiness; 3,000 students behind one college gateway report slowness. Why?** "They share one public IP, so IP hash sends them all to one server; nginx's ip_hash even uses only the first three octets of an IPv4 address, so a whole block maps together."

> **Where do you put sessions instead?** "A replicated in-memory store or database every server can read, about half a millisecond per lookup inside the data centre, or a signed token if I can live with harder revocation."

Weak: "Stateless means no database."

Strong: "State moved out of the server, and the stickiness failure modes."

Q6 "Music streaming servers are set on consistent hashing. What are the issues with it?"

CANDIDATE-REPORTED

Atlassian, Senior Software Engineer, scenario round (round 1); the post shows no interview date [S24].

Model answer. "Consistent hashing moves only about 1 over N of keys when servers change, which is why it is used. Its issues: hot keys, such as one viral song, all land on one server; without enough virtual nodes the ring is uneven; when a server dies, its neighbours inherit its load at once; nodes must agree on membership or keys route inconsistently; and moved keys still cause a burst of cache misses."

> **Why does mod-N not work?** "Going from N to N plus 1 servers keeps only 1 over N plus 1 of keys in place; from 9 to 10 servers, 90% of keys move and the cache hit rate collapses."

> **How uneven can it get without virtual nodes?** "In a 4-server simulation, 16% to 38% of keys per server with one point each, and 23% to 26% with 100 virtual nodes each."

> **One song gets 50 times normal traffic. Fix it.** "Detect hot keys and replicate them to several servers or to a CDN edge, or add a small local cache in front; consistent hashing alone balances keys, not per-key traffic."

Weak: "It has no issues, it is consistent." (The name is marketing; the hot key did not read it.)

Strong: "Hot keys, virtual nodes, membership, rebalancing, each with a fix."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 7 and 8

Q7 "A server behind the load balancer dies mid-request. What happens?" (and how do health checks work?) PATTERN-ONLY

None found for this wording. Context (Candidate-reported): a PhonePe Site Reliability Engineer candidate (interview October 2023, older than 24 months) answered "How do you ensure high availability in a distributed system?" with load balancing, redundancy, failover and health checks [S29].

Model answer. "Requests in flight on that server fail: the client sees a reset or a 502 or 504 from the balancer. New requests keep going to it until health checks notice, roughly threshold times interval, unless passive checks catch the first errors. An L7 balancer can retry idempotent requests on another server. For planned removals, connection draining avoids the failures entirely."

> **Why not retry every failed request?** "A POST may have done its work before the server died; retrying could charge twice. Retry only idempotent requests or ones with an idempotency key (Lesson 2)."

> **Interval 30 s, threshold 2, 857 requests per second to that server.** "About 60 seconds, about 51,000 requests; with 5-second checks about 10 seconds and 8,600."

> **Your health check queries the database. Good idea?** "Risky: a brief shared database problem fails every server's check and the balancer removes the whole fleet. Keep the check local; handle dependency failures with timeouts and circuit breakers, and cap how much capacity the automation may remove."

Weak: "The load balancer handles it."

Strong: "In-flight versus new requests, detection time, safe retries, draining."

Q8 Forward proxy vs reverse proxy; API gateway vs load balancer PATTERN-ONLY

None found.

Model answer. "A forward proxy sits in front of clients and makes requests for them, hiding them from servers, like an office web filter. A reverse proxy sits in front of servers and receives requests for them, hiding them from clients; load balancers, CDNs and API gateways are reverse proxies. A load balancer spreads load across copies of one service; an API gateway routes across different services and applies policies such as authentication and rate limits."

> **Why put a reverse proxy in front of an app at all?** "One public entry point: TLS termination, load balancing, caching, hiding origin IPs, and a single place for rate limits and logging."

> **Your app logs show every request from 10.0.0.5. Why?** "That is the proxy's address; read the client IP from X-Forwarded-For, and trust it only from your own proxies."

> **Gateway or load balancer for a 20-service backend?** "Both: a gateway at the edge for routing and policy, and load balancing for each service's copies; one product can do both."

Weak: "Reverse proxy is a proxy in reverse."

Strong: "Whom it acts for, examples, and how a gateway differs."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 9 and 10

Q9 What is a CDN, and when would you use one? PATTERN-ONLY

None found.

Model answer. "A CDN is a network of edge servers near users that cache content; on a miss the edge fetches from the origin and keeps a copy. It cuts latency and origin bandwidth and absorbs spikes. I use it for static, cacheable content such as images, scripts and video; personal and write traffic still goes to the origin."

> **Why is it faster?** "Shorter distance means shorter round trips; if the edge is 5 ms away instead of 30 ms, every round trip saves 25 ms."

> **2 TB of static data a day, 95% hit rate. Origin bandwidth?** "2 TB over 86,400 seconds is about 23 MB/s, about 185 Mbps; 5% of that is about 9 Mbps."

> **Users see old JavaScript after a deploy.** "The edge still holds the old copy; use versioned or content-hashed file names so a new deploy is a new URL, or purge. How long to cache is a Lesson 6 question."

Weak: "CDN makes websites fast."

Strong: "Edge, origin, miss path, numbers, and what not to cache."

Q10 Design (or implement) a load balancer CANDIDATE-REPORTED

Razorpay, Lead Software Engineer, Bangalore: a 2-hour machine-coding round, "Design Load Balancer", as a REST application with endpoints to add and remove backends, update timeouts and expose health checks, routing all other requests to mocked backends; the post gives months but no year [S23]. Microsoft SDE-2 2025 compilation: "Design a Load Balancer" [S25]. Outside the target list, DoorDash (PracHub, posted Jan 2026): fix a round-robin balancer, then implement consistent hashing with add_server, remove_server, get_server(key) [S30].

Model answer. "Core pieces: a registry of backends with weights and health status; a pluggable strategy interface (round robin, weighted, least connections, consistent hash) so algorithms can be swapped at runtime; a health checker that probes on an interval and flips status after thresholds; the request path that picks a healthy backend, forwards with a timeout, and retries idempotent requests once on another backend; and admin endpoints to add, remove and drain backends. Shared state such as the round-robin counter and connection counts must be thread-safe."

> **Why a strategy interface?** "Requirements change: the Razorpay prompt explicitly expected extensibility for algorithms and dynamic config, and the Strategy pattern (Lesson 14) lets me add one without touching the request path."

> **Two threads read the round-robin index at once.** "Both may pick the same backend and skip one; use an atomic increment and take the result modulo the healthy count."

> **One balancer is not enough.** "Run several balancer instances behind DNS or an L4 tier; for consistent hashing they must share the same membership list, or the same key routes to different servers."

Weak: "A list and a for loop."

Strong: "Registry, strategy, health checker, forwarding with timeouts and safe retries, admin API, concurrency."

INTERVIEW MODE: SAY IT LIKE THIS

- "For any of these questions I give the definition, one number that shows I understand it, the failure mode, and the fix."
- "A load balancer needs an algorithm, health checks and its own redundancy. DNS is cached for its TTL. Consistent hashing moves about one Nth of keys but does not fix hot keys. Forward proxies act for clients and reverse proxies act for servers."

HOW TO THINK WHEN IT BREAKS

Scenario questions 1 to 4

All scenarios are Pattern-only (no company source for these exact situations); numbers are original assumptions unless cited.

1. After adding a second app server, users say "the site keeps logging me out". PATTERN-ONLY

1. Ask what changed: one server became two behind round robin.
2. Predict: with 2 servers and in-memory sessions, about half of follow-up requests reach the wrong server; with 9 servers, about 89%.
3. Confirm: log the server ID on each request; logged-out requests should be on the server that did not create the session.
4. Fix: move sessions to a shared store or a signed token. Sticky sessions only as a short stopgap, and say why (uneven load, lost sessions).

The bug that only exists in production, because production is the only place with two servers.

2. One server is at 95% CPU while the others sit at 30%, with round robin. PATTERN-ONLY

1. Check whether the servers are identical. If one is smaller, use weights proportional to capacity.
2. Check request mix: are long requests (exports, uploads, WebSockets) landing unevenly? Count in-flight requests per server (Little's Law: rate x time).
3. Check stickiness: IP hash with a big NAT gateway or one large customer pins many users to one server.
4. Fix by cause: weights, least connections for uneven durations, or removing stickiness. Then re-measure.

3. You moved the service to a new IP an hour ago; some users still reach the old one. PATTERN-ONLY

1. Look up the record's TTL (dig sho.rt); if it was 3,600 s, caches may legitimately hold the old IP for up to an hour.
2. Keep the old IP serving (or proxying to the new one) until its traffic drops to near zero; some clients hold answers longer than the TTL.
3. Next time: lower the TTL to 60 s at least one old-TTL in advance, change the IP, then raise the TTL again.
4. Do not "fix" this by asking users to restart routers; fix the plan.

4. Health checks are green, but users get 500 errors from one server. PATTERN-ONLY

1. The check is too shallow: it tests "process answers", not "can serve real requests".
2. Confirm: compare per-server error rates in balancer logs; one backend stands out.
3. Add passive checks (mark a server failed on real errors, as nginx's max_fails does [S7]) and make the active check exercise a cheap real path.
4. Do not make the check depend on shared systems, or one database blip turns every check red (Scenario 5).

HOW TO THINK WHEN IT BREAKS

Scenario questions 5 to 8

5. A 20-second database blip, and the whole site went down for 5 minutes. PATTERN-ONLY

1. Timeline: the database blipped; every server's deep health check failed; the balancer removed all servers; recovery needed 5 consecutive successes per server.
2. Arithmetic: with interval 30 s and healthy threshold 5, a server needs about 5 x 30 = 150 s to return, after first being removed; restarts and warm-up add the rest.
3. Fix: shallow health checks, timeouts and circuit breakers for dependencies (Lesson 11), and a limit on how much capacity health checks may remove.
4. Cite the real precedent: in AWS's October 2025 incident, flapping health checks removed capacity until automatic failover was disabled [S1].

The health checks did their job perfectly. That was the problem.

6. You added a 10th cache-backed server and the database load jumped about 9x. PATTERN-ONLY

1. Suspect routing by hash(key) mod N: going from 9 to 10 servers moves 90% of keys, so 90% of cache lookups miss.
2. Check: hit rate per server dropped sharply at the moment of the change.
3. Switch to consistent hashing with virtual nodes (in nginx, hash $request_uri consistent; [S7]): about 10% of keys move.
4. Also add servers at quiet times, and pre-warm the new server if possible.

7. All requests in your app logs come from 10.0.0.5, and per-IP rate limiting blocks everyone at once. PATTERN-ONLY

1. 10.0.0.5 is the reverse proxy's address; the app sees the proxy, not the user.
2. Read the client address from X-Forwarded-For, which the proxy adds.
3. Trust that header only when the request came from your own proxy; otherwise anyone can fake an IP and dodge rate limits.
4. Re-test the rate limiter with real client IPs.

8. During every deploy, about 1% of requests fail for a minute. PATTERN-ONLY

1. The deploy stops servers while requests are in flight, and the balancer keeps sending new requests until health checks notice.
2. Fix the order: deregister first, let in-flight requests finish (connection draining; AWS deregistration delay, 300 s by default [S33], which many teams shorten when requests are short), then stop the process.
3. Start new servers, wait until they pass the readiness check, then add them.
4. Roll one zone or a few servers at a time, so capacity never drops below what the peak needs (Concept 1: losing a third of 9 servers still leaves 64% load).

HOW TO THINK WHEN IT BREAKS

Scenario question 9 and the pattern

9. Users in one city report the site is slow; the servers look idle. PATTERN-ONLY

1. Separate network from server time: compare server-side latency (low) with client-side timings.
2. Check DNS: is that city's resolver sending users to a far-away region or an old address?
3. Check the CDN: are static files served from a nearby edge, or missing and going to the origin every time?
4. Remember Uber's finding: extra proxy hops in more regions did not consistently reduce latency [S19]. Measure the path; do not assume more boxes means faster.

THE PATTERN BEHIND EVERY SCENARIO

When something breaks, **ask what changed**, **predict the symptom with a number**, **confirm** it with per-server logs or metrics, and only then **fix the cause**.

INTERVIEW MODE: SAY IT LIKE THIS

- "Common causes in this area are in-memory state, uneven requests under round robin, DNS caching, health checks that are too shallow or too deep, mod-N remapping, and trusting the wrong client IP."

SYMPTOM	FIRST SUSPECT	CONCEPT
Random logouts after scaling out	Sessions in server memory	2
One hot server under round robin	Unequal servers, long requests, IP stickiness	3, 4, 2
Old IP still getting traffic	DNS TTL and caching	8
Green checks, red errors	Health check too shallow	7
Whole site down after a DB blip	Health check too deep	7
DB load jumps after adding a server	mod-N remapping	5
Everyone has the same IP	Reading the proxy's address, not X-Forwarded-For	9
Errors on every deploy	No draining	7
One city slow, servers idle	DNS routing, CDN misses, extra hops	8, 10

SOLVE THESE BEFORE YOU LOOK

Practice problems 1 to 3

Original problems, not reported questions. Cover the solution, try it on paper, then check.

P1 Forward or reverse? EASY

Classify: (a) a college network's web filter; (b) nginx in front of three app servers; (c) a CDN edge server; (d) a privacy service your browser sends all traffic through; (e) an API gateway.

> **Hint.** Ask: does it act for the clients or for the servers?

SOLUTION

(a) Forward: it acts for the students. (b) Reverse: it acts for the servers. (c) Reverse: it serves the site's content for the origin. (d) Forward: it acts for you. (e) Reverse: it receives requests for the backend services.

P2 Weights and load EASY, NUMERICAL

Three servers with capacities 1,000, 1,000 and 3,000 requests per second. Traffic is 4,000 requests per second. (a) Load on each with plain round robin? (b) Choose weights and give the load on each. (c) Which server is the bottleneck in (a)?

> **Hint.** Share = weight / sum of weights. Load % = traffic to the server / its capacity.

SOLUTION

(a) Each gets 4,000 / 3 = about 1,333 per second: 133% on the two small servers (overloaded) and 44% on the big one. (b) Weights 1 : 1 : 3 (sum 5): 800, 800 and 2,400 per second, which is 80% on every server. (c) The two small servers, which are over capacity.

P3 Health-check arithmetic NUMERICAL

A service has 12 servers at 9,600 requests per second total. One server dies. (a) Requests per second to the dead server? (b) Requests sent to it before removal with interval 30 s and unhealthy threshold 2? (c) With interval 5 s and threshold 3? (d) With healthy threshold 5 and interval 30 s, how long before a recovered server gets traffic?

> **Hint.** Detection = about threshold x interval. Requests = rate x time.

SOLUTION

(a) 9,600 / 12 = 800 per second. (b) About 2 x 30 = 60 s; 800 x 60 = 48,000. (c) About 3 x 5 = 15 s; 800 x 15 = 12,000. (d) About 5 x 30 = 150 s. (All "about", because a failure can begin anywhere inside an interval and probes can wait up to the timeout.)

SOLVE THESE BEFORE YOU LOOK

Practice problems 4 and 5

P4 mod-N versus consistent hashing NUMERICAL

A cache layer routes 1,000,000 keys. (a) Going from 4 to 5 servers with mod-N, how many keys move? (b) Going from 10 to 11? (c) Roughly how many move with consistent hashing in each case? (d) Going from 5 servers down to 4 with mod-N, what fraction stays?

> **Hint.** From N to N + 1 with mod-N, the fraction that stays is 1 / (N + 1). With consistent hashing about 1 / (new N) moves.

SOLUTION

(a) Stay: 1 / 5 = 20%, so 800,000 move. (b) Stay: 1 / 11 = about 9.1%, so about 909,000 move. (c) About 1 / 5 x 1,000,000 = 200,000, and about 1 / 11 x 1,000,000 = about 91,000. (d) Going 5 -> 4 is the reverse of 4 -> 5: the keys with h mod 4 = h mod 5, which is 1 / 5 = 20%. So 80% move either way.

P5 Plan a DNS migration MEDIUM

api.sho.rt has an A record with TTL 86,400 s (one day). You must move to a new load balancer IP on Saturday at 02:00 with minimal disruption. (a) When must you lower the TTL, and to what? (b) After the switch, how long should the old IP keep serving? (c) What is the cost of leaving the TTL at 60 s forever?

> **Hint.** Caches keep the old TTL value until it expires, so a TTL change itself takes up to one old TTL to spread.

SOLUTION

(a) Lower it to 60 s (for example) at least 24 hours before, by Friday 02:00 at the latest; earlier is safer. (b) At least a few TTLs after the switch, and until traffic on the old IP is near zero, because some clients hold answers longer than the TTL. (c) More DNS queries (each resolver asks 86,400 / 60 = 1,440 times a day instead of once) and an extra lookup delay for users whose cache expired; raise it again after the migration, say to 300 to 3,600 s.

SOLVE THESE BEFORE YOU LOOK

Practice problem 6

P6 Survive a zone failure, and choose the balancer HARD

A payment-status API ("Has my UPI payment gone through?") peaks at 18,000 requests per second. Each server handles 2,500. Requests are short HTTP calls; some clients keep long polling connections for up to 30 s. You must survive the loss of one of 3 zones at no more than 75% load per server. (a) Minimum servers per zone? (b) Load per server at peak with all zones up? (c) L4 or L7, and which algorithm? (d) Health-check settings so a dead server receives under 10,000 requests. (e) Is IP-hash stickiness acceptable here?

Hint. Size for two zones carrying everything at 75% of 2,500. Long-held connections matter for the algorithm choice.

SOLUTION

(a) Two zones must carry 18,000 at 75% x 2,500 = 1,875 per server: 18,000 / 1,875 = 9.6, so 10 servers across the two surviving zones, which is 5 per zone, and 15 in total. (b) 18,000 / 15 = 1,200 per server, 48%. After losing a zone: 18,000 / 10 = 1,800, which is 72% (under 75%). (c) L7, because it is HTTP and we want path-level routing, TLS termination and safe retries of the idempotent status GET; least connections (or least-pending requests), because long polling connections make request counts a poor measure of load. (d) Each server gets 1,200 per second; to stay under 10,000 requests, detection must be under 10,000 / 1,200 = about 8.3 s, for example interval 4 s and threshold 2 (about 8 s; 5 s is AWS's minimum interval [S32], so on AWS use 5 s x 2 = 10 s and rely on passive checks and retries to cover the gap). (e) No: the API should be stateless; stickiness would pile users from big NAT gateways onto single servers and break zone failover.

HOW TO CHECK YOUR OWN ANSWER

Write the assumptions first. Then one line per step, each with its unit. Then the failure case (a zone, a server, a hot key). Then the decision. If any of the four is missing, the answer is incomplete, even if the number is right.

TEN QUESTIONS, TWO MINUTES EACH

Mini quiz

Original questions. Cover the answers, answer all ten, then check.

1. Which change is horizontal scaling?
A) 8 to 32 CPU cores on the same server B) Adding 4 more servers behind a load balancer C) Faster SSDs D) Adding RAM
Answer B. Horizontal means more machines. A, C and D make one machine bigger (vertical).

2. nginx weights 3, 1, 1. Share for the weight-3 server?
A) 33% B) 50% C) 60% D) 75%
Answer C [S6]. 3 / (3 + 1 + 1) = 60%. A ignores weights. B and D are not weight / total.

3. Why does least connections help when one server becomes slow?
A) It restarts the slow server B) The slow server holds requests longer, so its in-flight count rises and it gets fewer new requests C) It caches responses D) It uses the client IP
Answer B. In flight = rate x time (Little's Law). A is not a balancer's job. C is caching. D describes IP hash.

4. With hash(key) mod N, 9 to 10 servers moves about what share of keys?
A) 10% B) 50% C) 90% D) 100%
Answer C. Only 1 / 10 of keys stay. A is what consistent hashing moves. B has no basis. D overstates: some keys stay.

5. What do virtual nodes fix in consistent hashing?
A) Hot keys B) Uneven share of the ring per server C) DNS caching D) TLS overhead
Answer B [S14]. Many points per server even out each server's share. A is not fixed (one key still maps to one server). C and D are unrelated.

6. Which can route /api/* and /images/* to different pools?
A) An L4 load balancer B) An L7 load balancer C) Round-robin DNS D) A forward proxy
Answer B [S8, S12]. Only L7 reads the HTTP path. A sees IPs and ports. C returns addresses, not paths. D acts for clients.

7. Interval 30 s, unhealthy threshold 2. Roughly how long until a dead server stops getting traffic?
A) 2 s B) 15 s C) 60 s D) 150 s
Answer C [S31, S32]. About threshold x interval = 2 x 30 s. D is the recovery time with healthy threshold 5. A and B ignore the interval.

8. In a full DNS lookup, which server returns the domain's actual A record?
A) Root B) TLD C) Authoritative D) The stub resolver
Answer C [S2]. Root refers to the TLD; the TLD refers to the authoritative server, which holds the record. D only forwards the question.

9. A record has TTL 3,600 s. You change its IP. What is true?
A) Everyone sees the new IP instantly B) Caches may serve the old IP for up to about an hour C) The root servers must be updated D) TTL only affects email
Answer B [S3]. TTL is how long an answer may be cached. A ignores caching. C: roots do not hold your records. D is false.

10. An office proxy that filters which websites employees can visit is a:
A) Reverse proxy B) Forward proxy C) CDN D) Load balancer
Answer B [S4]. It acts for the clients (employees). A, C and D act for servers.

FIVE SENTENCES TO KEEP

Recap and the 60-second answer

1. Vertical scaling buys a bigger machine (simple, capped, a single point of failure); horizontal scaling adds machines behind a load balancer and requires stateless servers.
2. Algorithms: round robin for equal servers and requests, weights for unequal servers, least connections for uneven or long requests, hashing for affinity; consistent hashing moves about 1/N of keys instead of N/(N+1).
3. L4 balancers route on IPs and ports; L7 balancers are HTTP-aware reverse proxies that route by path or header, terminate TLS and retry.
4. Health checks remove dead servers after about threshold x interval; keep them shallow, cap what they can remove, retry only idempotent requests, and make the balancer itself redundant.
5. DNS: stub -> recursive resolver -> root -> TLD -> authoritative, cached for the TTL; forward proxies act for clients, reverse proxies (load balancers, gateways, CDNs) act for servers; CDNs cache static content at the edge.

SAY THIS IN AN INTERVIEW: "HOW WOULD YOU SCALE THIS SERVICE TO HANDLE MUCH MORE TRAFFIC?"

"First I keep the app servers stateless, with sessions in a shared store or a signed token, so I can scale horizontally instead of buying an ever-bigger machine. I put an L7 load balancer in front, which terminates TLS and routes by path; I use round robin or weighted round robin for uniform traffic, least connections where requests vary, and consistent hashing where I want cache affinity, because it moves only about one Nth of keys when servers change. Health checks every few seconds remove a dead server in about ten seconds, I retry only idempotent requests, and the balancer runs redundantly across zones, sized so losing a zone stays under about 75% load. DNS returns the balancer addresses with a moderate TTL, which I lower before any migration. Static content goes on a CDN so the origin serves only dynamic traffic. Then I check the next bottleneck, usually the database and the cache, which are the next two lessons."

SUB-TOPIC (STUDENT CHECKLIST)	COVERED HERE	WHERE ELSE
Vertical vs horizontal scaling	Fully, with availability arithmetic	Lesson 7 (scaling databases)
Stateless servers, sticky sessions	Fully	Lesson 6 (session store as a cache)
Round robin, weighted, least connections, IP hash	Fully	Lesson 15 (machine-coding a balancer)
Consistent hashing	Fully as a routing tool	Lesson 7 (sharding), Lesson 31 (key-value store)
L4 vs L7	Fully	None needed
Health checks, failover, draining, LB redundancy	Fully	Lesson 11 (reliability)
DNS resolution, TTL, DNS load balancing	Fully	Lesson 2 (URL journey)
Forward vs reverse proxy, API gateway	Fully	Lesson 25 (rate limiter at the gateway)
CDN basics	Basics	Lesson 6 (cache lifetimes, invalidation)

CHECK OUR WORK

Sources (1 of 2)

S = opened in full; X = seen only in search extracts. Every calculation in this lesson was recomputed in code when this PDF was built.

ID	PAGE	URL
S1	AWS: Summary of the Amazon DynamoDB Service Disruption in US-EAST-1 (Oct 2025)	aws.amazon.com/message/101925
S2	Cloudflare Learning: DNS server types	cloudflare.com/learning/dns/dns-server-types
S3	RFC 1034: Domain Names, Concepts and Facilities	rfc-editor.org/rfc/rfc1034.html
S4	Cloudflare Learning: What is a reverse proxy?	cloudflare.com/learning/cdn/glossary/reverse-proxy/
S5	Cloudflare Learning: What is a CDN edge server?	cloudflare.com/learning/cdn/glossary/edge-server
S6	nginx docs: Using nginx as HTTP load balancer (DevDocs-style mirror)	docs.w3cub.com/nginx/http/load_balancing
S7	nginx docs: ngx_http_upstream_module	nginx.org/en/docs/http/ngx_http_upstream_module.html
S8	HAProxy blog: Load balancing FAQ (9 Jun 2011, modified Dec 2025)	haproxy.com/blog/loadbalancing-faq/
S9	HAProxy Enterprise docs: Load balancing algorithms	haproxy.com/documentation/hapee/latest/load-balancing/load-balancing-algorithms/
S10	Cloudflare Learning: Types of load balancing algorithms	cloudflare.com/learning/performance/types-of-load-balancing-algorithms/
S11	Cloudflare Learning: What is DNS-based load balancing?	cloudflare.com/learning/dns/dns-load-balancing/
S12	AWS Elastic Load Balancing API Reference: Welcome (23 Sep 2026)	docs.aws.amazon.com/elasticloadbalancing/latest/APIReference/Welcome.md
S13	AWS CloudFormation: AWS::ElasticLoadBalancingV2::TargetGroup	docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-resource-elasticloadbalancingv2-targetgroup.md
S14	Karger et al.: Consistent Hashing and Random Trees (paper PDF)	cs.brown.edu/courses/csci2950-u/f09/papers/chash97stoc.pdf
S15	The Twelve-Factor App: VI. Processes	12factor.net/processes
S16	AWS Well-Architected, Reliability Pillar: Design principles	docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/design-principles.html
S17	Google Research: Maglev, A Fast and Reliable Software Network Load Balancer	research.google/pubs/pub44824/
S18	Uber Engineering: Better Load Balancing: Real-Time Dynamic Subsetting (17 May 2022)	uber.com/en-BR/blog/better-load-balancing-real-time-dynamic-subsetting/
S19	Google Cloud blog: Uber's modern edge (14 Aug 2025)	cloud.google.com/blog/products/networking/ubers-modern-edge-a-paradigm-shift-in-network-performance-and-efficiency
S20	Glassdoor: Amazon Software Engineer, Chennai (interview Apr 2025, posted 20 Feb 2026)	glassdoor.es/Entrevista/Amazon-Entrevista-E6036-RVW102890171.htm
S21	Glassdoor: Google Customer Solutions Engineer, Singapore (8 Jan 2026)	glassdoor.co.uk/Interview/Google-Interview-E9079-RVW23271044.htm

CHECK OUR WORK

Sources (2 of 2)

ID	PAGE	URL
S22	TechPrep: Adobe interview process (2026), aggregator	techprep.app/companies/adobe
S23	LeetCode: Razorpay Lead Software Engineer, Bangalore (no year shown)	leetcode.com/discuss/post/7360702/razorpay-lead-software-engineer-bangalor-qkqi/
S24	LeetCode: Atlassian Senior Software Engineer, offer (undated)	leetcode.com/discuss/interview-experience/6344788/Atlassian-or-Senior-Software-Engineer-or-Offer/
S25	LeetCode: Microsoft SDE-2 Recent questions 2025, consolidated	leetcode.com/discuss/interview-question/6403987/Microsoft-SDE-2-Recent-questions-2025-or-Consolidated/
S26	LeetCode: Zomato SDE Off-Campus 2025 (no post date shown)	leetcode.com/discuss/post/6493240/zomato-interview-experience-sde-off-campus-2025
S27	LeetCode: Flipkart SDE 2 interview experience (months given, no year)	leetcode.com/discuss/post/7591327
S28	Glassdoor: Amazon SDE-2, Chennai (interview Apr 2024)	glassdoor.co.uk/Interview/Amazon-Interview-E6036-RVW98587522.htm
S29	Glassdoor: PhonePe Site Reliability Engineer reviews (incl. Oct 2023 interview)	glassdoor.fr/Entretien/PhonePe-Entretien-E2289138-RVW91851846.htm
S30	PracHub: DoorDash, fix a round-robin LB, then consistent hashing (18 Jan 2026)	prachub.com/interview-questions/fix-a-round-robin-load-balancer-then-implement-consistent-hashing
S31	AWS docs: Health checks for Application Load Balancer target groups	docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html
S32	AWS docs: Health checks for Network Load Balancer target groups	docs.aws.amazon.com/elasticloadbalancing/latest/network/target-group-health-checks.html
S33	AWS docs: Edit target group attributes for your Application Load Balancer	docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-target-group-attributes.html
S34	nginx docs: ngx_http_proxy_module (embedded variables)	nginx.org/en/docs/http/ngx_http_proxy_module.html
X1	The consistent-hashing paper's venue: STOC 1997 (course-site file title; search extract only)	courses.e-ce.uth.gr/CE623/STOC_1997_Consistent_hashing.pdf

Honest gaps: things interviewers sometimes reach that this lesson only touches or leaves out

- Anycast and BGP routing in depth; GeoDNS and latency-based DNS policies (named only)
- DNS security (DNSSEC, DNS over HTTPS, cache poisoning) and record types beyond A, AAAA, CNAME, NS (MX, TXT, SRV)
- Advanced algorithms: power of two random choices, weighted response time, resource-based, Maglev's lookup-table hashing, rendezvous hashing
- Global server load balancing, multi-region active-active failover; direct server return, NAT and tunnelling modes of L4 balancers [S8]
- TLS at the balancer: certificates, SNI, re-encryption, mutual TLS
- Autoscaling policies; Kubernetes Services, Ingress, service meshes (named only)
- DDoS protection, WAFs, edge rate limiting (Lessons 18 and 25)
- WebSockets and long-lived connections behind balancers (touched in Concept 4)
- CDN internals: cache keys, Cache-Control, purge, origin shield (Lesson 6)

Company evidence: no reports found for Uber, Swiggy or PhonePe software-engineering rounds on these topics inside the window; the Google report is a solutions-engineering role. Also opened, but nothing on these topics: PracHub load-balancing topic pages, LeetCode PhonePe Backend SE (Oct 2025), Glassdoor Swiggy, Zomato, Uber and Microsoft SDE reviews, Taro Flipkart SDE pages, a LeetCode Amazon "last 6 months" list. Original work: the 100,000-key mod-N and consistent-hashing simulation, all sho.rt numbers beyond Lesson 4's, the practice problems and the quiz. Not claimed: that nothing else can be asked.

SELF-CHECK

Can you say yes to all four?

- **I can choose up or out**
  And say why horizontal needs stateless servers.
- **I can pick and defend an algorithm**
  Round robin, weights, least connections, consistent hashing.
- **I can size health checks**
  Detection = threshold x interval, and the deep-check trap.
- **I can walk DNS and proxies**
  Stub to authoritative, TTL, forward vs reverse, CDN.

> "How does a click reach your server?" **Now you answer outside-in: DNS, CDN, balancer, stateless servers, and what breaks at each step.**

YOUR NEXT STEPS

STEP 01	STEP 02	STEP 03
Solve problems 1 to 6 on paper	Run dig +trace on any site	Read ahead: Lesson 6, caching